knowledge_type: book
title: Mars Person Reference Handbook
title_zh: Mars 人员参考手册
client: mars
edition: "2026.1"
author: 数砚公司
classification: 数砚机密
handling: 仅限数砚公司内部授权人员使用。未经数砚公司书面许可，禁止以任何形式对外复制、传播、披露或提供。非授权人员如误获本文件，应立即停止使用，并永久删除或销毁其持有、存储或控制的全部副本（含打印件、缓存及衍生副本）。违反上述规定的，数砚公司将依据适用法律、合同约定及公司制度追究相关人员责任。
status: published
last_updated: 2026-10-27
generated_by: tools/ps/build_reference_handbook_editions.py
source_manifest: editorial/manuscript-manifest.json
source_sha256:
"appendix/a-person-objects-fields-and-relations.md":
"edd131ecd809b95306e836ab55e9c80af9ad018b36d12043e3c6eaff4ccad4dc"
"appendix/b-person-systems-and-data-lines.md":
"a12c899873f244329d24995dc86b05efafcf0e59f147871e40d87fe410e03c69"
"appendix/c-person-project-and-access-workbench.md":
"c0ea5e1ddc5f3690d225b732a559739b76d156b7ae88d2678abf1df7b7151ece"
"appendix/d-person-open-questions-privacy-and-maintenance.md":
"1025a6cbb8be719437e721796f899ac35ed229519e01ff2f778e48e5f2e35dfe"
"book/01-person-is-not-a-user.md":
"0d12a1308147d665fcae81ad9671df5ac63008bc25d3592d3d662836dba3c314"
"book/02-who-is-a-business-execution-person.md":
"4613f2f29639e380cc6c583b54a7259835124e51f74ab74d0bda92e7e4d55bf9"
"book/03-where-person-identity-comes-from.md":
"588a84b0fddb7a3ab93eca5fba485a9304ec5cd67f888c8021afc705b12fc4bc"
"book/04-organization-supervisor-and-geography.md":
"cff06b2435df788837b01cf11b079566354ce5c2576c1c9cdef8479c4c49c7bb"
"book/05-user-account-login-and-third-party-identity.md":
"b00857f5b427fc12ab7aa234edf8ead308cf38097e99d8c195d398918169e527"
"book/06-application-client-role-menu-permissions.md":
"d0edb6640ccf7e9c0280c48a0b5ae8a44c9843e681264bd8d3d2126e05fe58b1"
"book/07-data-permissions.md":
"cd5f274e31b137f59f762b2894002dae348277c85382387ec17de7a9297a0cfa"
"book/08-store-person-relations.md":
"922357664dd2759aada837181e07e7c268c7831aff5bb5355342d08a3976a2e8"
"book/09-supervisor-and-other-business-relations.md":
"7501429f50b19c4c66ce1c4e8e45ba5775a9934119ab4d0a00e9894eb1cf6e36"
"book/10-people-in-visits-approvals-ttf-and-reports.md":
"b5458ddb0fe2e79d07e9b1ade2b454b7eeb06db66ede7c054b617bbcde11ab84"
"book/11-joiner-mover-leaver-identity-access-relations.md":
"c30cb9ca840a5dcdcd513cdcb02872af933f2e0ecf95c7a0d9e6e2a60e60fa77"

"book/12-person-change-impact-project-access-and-operations.md":
"9bc2e3a0e681e0da970325194b228315fdba88ea269a1310f7a272a0530f22f3"
"editorial/concept-registry.json":
"dea93944def52f6842a51069f8c26f2ef6a1e5b8f6d9e190aed737d81803e044"

Mars 人员参考手册

作者： 数砚公司

密级： 数砚机密

使用限制： 仅限数砚公司内部授权人员使用。未经数砚公司书面许可，禁止以任何形式对外复制、传播、披露或提供。非授权人员如误获本文件，应立即停止使用，并永久删除或销毁其持有、存储或控制的全部副本（含打印件、缓存及衍生副本）。违反上述规定的，数砚公司将依据适用法律、合同约定及公司制度追究相关人员责任。

版本 `2026.1` **状态** 证据样稿（已发布）

人员、账号、组织、权限、门店责任与业务消费

阅读契约

先按第 1–12 章建立身份、账权、组织、关系、下游消费和生命周期主线，再用 A–D 案头卷查字段、系统链、访问工作台和隐私缺口。

本装配稿由章节和案头卷确定性生成。单项事实仍以各章 source_docs、claim ledger、文档、代码、staging 部署与 staging database 为准。

目录

- 第 1 章 人员不是用户，也不是角色
- 第 2 章 谁算业务执行人员：员工、外包、DA、DSR 与经销商人员
- 第 3 章 人员身份从哪里来：MDP、用户中心与加工副本
- 第 4 章 组织、主管链与玛氏地理
- 第 5 章 用户、账号、登录与第三方身份
- 第 6 章 应用、客户端、角色与菜单权限
- 第 7 章 数据权限：组织、资源与门店范围
- 第 8 章 人店关系：业务员、门店负责人、来源与生效时间
- 第 9 章 主管链与其它业务关系
- 第 10 章 人员怎样进入拜访、审批、ttf 和报表
- 第 11 章 入转调离、账号、权限与关系生命周期
- 第 12 章 人员变更影响、项目开通与运维演进
- 第 A 卷 人员对象、字段与关系
- 第 B 卷 人员系统、身份、权限与数据链
- 第 C 卷 人员项目与访问工作台
- 第 D 卷 人员事实缺口、隐私与维护

概念与别名速查

本表由 `editorial/concept-registry.json` 生成，用于检索规范名、英文名和别名；正文仍须首次使用时解释概念。

规范名	英文名与别名	不等于	最短定义
人员	Person / Personnel / Personnel / 员工	用户账号、角色、项目成员	企业人员或进入 Mars 业务链的第三方执行人员对象。
用户账号	User Account / user / user_info / 账户	人员在职、角色	某应用下与工号关联的登录账户。
第三方身份绑定	Third-party Identity Binding / OKTA binding / EE_WECHAT binding	业务权限、项目成员	外部 IdP 身份与应用工号之间的映射。
组织节点	Organization Node / organization_content / org content	门店责任、数据资源	组织树中的可用层级节点。
自身组织	Own Organization / user_org / own organization	授权组织	用户作为组织归属持有的节点。
授权组织	Granted Organization / grant_user_org / grant org	业务责任人、自身组织	为明确业务目的额外授予用户访问的组织节点。
业务角色	Business Role / role / 角色	用户、数据范围	在应用内组合功能与权限标志的角色对象。
客户端	Client / client / 登录入口	应用、菜单	应用内的具体登录与页面入口。
菜单	Menu / menu / 功能菜单	数据权限	客户端内可访问的功能节点。
数据资源	Data Resource / resource_content / 资源节点	组织节点、业务事实	用于数据权限展开的资源内容节点。
主管链	Supervisor Relation / 主管关系 / reporting line	人店关系、项目经理	人员向谁汇报的期次化关系。
人店关系	Store-Person Relation / store_emp / 门店人员关系	主管链、线店关系	门店、人员、职责、来源、状态和有效期共同构成的责任关系。
门店职责	Store Duty / duty_type_code / L / F	角色、人员类型	人员对门店承担业务员或负责人等职责的代码。
线店关系	Route-Store Relation / store_route_binding / 线路门店绑定	人店责任、拜访计划	门店所属线路及其顺序和有效期。
项目成员	Project Member / members.yaml / 项目角色	业务账号、人店责任	项目仓登记的自然人责任和参与阶段。
人员投影	Person Projection / user_synthetic / store_query	人员事实源、业务权限	面向查询、计划或报表生成的人员/关系派生值。
合作方人员关系	Cooperator User Relation / org_cooperate_user / cooperate_user_mapping	Mars employee、project member	外部合作方人员与组织/系统的关系及其映射。
经销商人员映射	Distributor Employee Mapping / distributor_emp / distributor_emp_da	Mars employee、customer relation	经销商侧人员与业务编码的映射，分布在多个系统副本。

第 1 章 人员不是用户，也不是角色

项目启动时，负责人说“把张三加到系统里”。这句话可能同时包含四件事：新增企业人员、给他在某个应用创建账号、授予客户端与角色菜单、再把他绑定到负责门店。任何一件遗漏，用户看到的症状都可能只是“进不去”或“看不到”。

如果团队把项目成员名册当业务账号，把登录成功当权限完成，或把门店责任人姓名当一条有效关系，就会在不同系统之间来回补数据。每一步看起来都成功，业务结果仍可能不成立。

本章只回答一个问题：**当我们说“这个人”时，说的是人员、账号、权限还是业务关系？**

为说明边界，使用一个匿名复合案例：一位新同事能登录员工门户，却看不到目标业务应用；管理员给他加了角色，他仍只看到部分门店；门店页面显示负责人姓名，但审批人解析使用的是另一条主管链。案例不对应任何单一真实项目。它不是某个页面的故障，而是四个事实层尚未同时完成。

1. 四个事实层

事实层	回答	常见证据
企业人员	这个人是谁、是否在职、属于什么人员类型	人员主数据
应用账号	这个人用什么身份登录哪个应用	用户、第三方身份、账号状态
业务权限	这个人能使用哪些功能、看哪些数据	应用、客户端、角色、菜单、组织、资源
业务关系	这个人负责哪些门店、向谁汇报或关联哪个客户	人店关系、主管链、经销商人员映射

四层都需要稳定人员键或账号键连接，但任何一层都不能替代另一层。

账号存在不代表人员在岗，人员在岗也不代表已开通目标应用；登录成功不代表有正确数据范围，人店责任关系也不代表它在目标时点有效。把这几层分开检查，才能解释同一个“看不到”为什么可能来自完全不同的断点。

2. 人员记录和账号记录在代码里就是不同对象

当前本地 `re-yhx-master` 分支 `main`、HEAD `d52769a` 中，`PersonnelDo` 对应 `personnel` 表，声明了 `employeeCode`、人员姓名、人员类型、来源、经销商信息、上级主管、`onJob`、入离职日期、有效起止时间、变更来源和请求标识等字段。相比之下，`UserInfoDo` 对应 `user_info` 表，声明了 `appCode`、`employeeCode`、用户来源、冻结 `frozen`、有效 `valid`、账号 `account`、联系方式、客户端列表等字段；代码注释明确写着，用户有效无效看 `valid`。

这两段代码已经说明：企业人员档案与应用账号分别存储、分别有状态。它们通过人员键关联，但不是同一条记录的两个名字。

这种分离不是技术冗余。同一个人可能进入多个应用，不同应用账号可能有不同状态和客户端；一个人员记录即使姓名为空或已离职，账号仍可能有历史；一个账号即使有效，也可能尚未获得任何业务角色。

3. staging 行数不能直接回答“有多少人”

2026-10-26 22:48:35 CST，staging `yhx_master` 的精确聚合为：

- `user_info` 9,009 行；
- `personnel` 10,445 行；
- `mdp_personnel` 53,757 行；
- `organization` 7 行；
- `organization_content` 8,880 行；
- `role` 49 行；
- `menu` 260 行。

这些数字说明多类载体都有数据，却不能被相减或相除来解释“重复人数”“在职率”或“权限覆盖率”。不同表的保留策略、历史行、人员来源和状态过滤不同；本次查询也只做了总行数。

如果要回答“一个应用有多少有效账号”，至少还要按应用、`valid`、`frozen`、有效期和人员映射过滤。要回答“有多少人在职”，又必须选择人员主数据层和相应口径。问题不能从一个总数直接跳过去。

4. 角色/菜单是功能权限，不是人员属性

用户中心访问控制模型把应用、客户端、角色、菜单、组织、资源和授权分开。用户与角色之间、角色与客户端及菜单之间是不同关系。

“给一个人加角色”也不等于已经完成数据权限。角色回答功能入口，组织与资源回答数据范围。一个用户拥有某个页面菜单，仍可能看不到目标门店；能查询门店，也未必能编辑。

代码中的 `RoleDo` 对应 `role` 表，并声明应用、角色编码、名称、超级管理员标记、授权销售组织标记、授权配置测试标记和状态。它描述的是一个可复用角色，不是某个人的身份。

项目因此至少需要两类正负样本：

1. 功能权限：目标用户能看到目标菜单，看不到未授权菜单；
2. 数据权限：目标用户能看到范围内门店，看不到范围外门店。

登录成功只证明认证通过，不能替代这两类验收。

5. 自身组织和授权组织会合并成数据范围

当前 `UserOrganizationBizServiceImpl` 中，用户组织与授权组织的层级查询会分别取自身组织和授权组织，再把两边的组织内容 UID 合并到同一个层级结果。这与访问控制文档中的“自身组织 ∪ 授权组织”一致；解释一个用户能看哪些数据时，只看任何一边都会遗漏另一部分归属。

但“能解析到组织资源”仍然只是数据范围的一个前提。业务分群、销售地理、覆盖模式、门店过滤和入口鉴权可能继续收敛范围。最终验收仍要在目标应用和真实用户上观察范围内可见、范围外不可见。

6. 离职不是只改一个人员状态

当前 `PersonnelManageServiceImpl.update` 在 `onJob=0` 时，会同步写人员及其扩展信息中的离职状态和离职日期，然后提前返回，不继续执行普通编辑字段的更新分支。

这证明“离职”在人员侧有专门处理语义，不能当普通属性随便编辑。但它没有独自证明账号、角色、授权组织和人店关系都已回收。

知识库中的访问开通与人员规则将离职、权限回收和关系解绑视为链式影响。手册在描述离职时必须分别检查：

- 人员是否已标记离职；
- 应用账号是否冻结或失效；
- 角色和客户端是否回收；
- 自身组织和授权组织是否解除；
- 人店关系是否在正确日期关闭；
- 投影和下游报表何时刷新。

任何一个完成都不代表全部完成。

7. 人店关系是关系事实，不是“门店负责人”一列

staging `store_ops.store_emp` 在 2026-10-26 当前时间窗的聚合中，同一 `store_code + emp_code + duty_type_code` 之外还存在 `source`、`state` 和有效起止时间。

当次观察到 COT、DA、IC 三类来源，`L` 表示业务员、`F` 表示门店负责人，状态同时包含有效、审批中以及极少量其它状态。缺任一维都可能把审批中当成有效，把生效前的未来行当成当前，或把不同来源的关系误判为重复。

因此，“这家店属于谁”不是只查一个姓名：

```
store_code
+ duty_type_code
+ source
+ state
+ effective_start_timestamp / effective_end_timestamp
```

当前 staging 中，COT 的 `state=2` 关系在查询时间窗内仍有 686 行。它们等待审批或处于流程状态，不能因为时间窗覆盖就算当前有效责任。少量 `state=0` 和 `state=4` 也需要先查业务含义和来源，不能自行归并为“禁用”。

8. 一个人员问题应该按什么顺序排查

遇到“这个人不对”时，先按以下顺序定位：

1. 人员是否存在，在职状态和人员类型是什么；
2. 目标应用是否有账号，账号 `valid`、`frozen` 和有效期是什么；
3. 账号是否关联正确客户端、角色和菜单；
4. 自身组织、授权组织和资源范围是否覆盖目标数据；
5. 如果问题涉及门店责任，检查人店关系的来源、职责、状态和有效期；
6. 如果涉及审批人，检查对应期次的主管链，不用当前汇报关系猜历史；
7. 如果报表仍显示旧人，检查综合信息、查询投影和报表刷新时点；
8. 如果问题涉及离职或转岗，按人员、账号、权限、关系、投影、下游逐层验收。

这条顺序的价值在于阻止“加个角色再看看”。角色只能改变功能权限，不能修复人员映射、数据范围或关系时点。

9. 本章明确不知道什么

本章没有确认：

- 9,009 个用户账号与 10,445/53,757 个人员记录之间有多少一对一映射；
- 当前 staging 账号中有多少可登录、有多少有业务角色、有多少能解析到门店范围；
- `state=0/4` 在当前所有业务入口中的完整含义；
- 主管链与当前人店关系是否同源、同时点；
- 本地代码 HEAD 与 staging 镜像的精确提交对应；
- 生产环境是否具有相同数量和状态分布。

这些都进入后续章节和案头 D。下一章先把“谁算业务执行人员”讲清，再进入来源、组织和权限链。

第 2 章 谁算业务执行人员：员工、外包、DA、DSR 与经销商人员

当项目说“需要把执行人员纳入系统”时，最先出现的不是字段，而是范围：正式员工、DSR、品牌理货员、DA、BM 外包、第三方供应商、经销商业务员，哪些需要账号，哪些只需要关系，哪些只出现在报表?

这些称谓来自不同维度。有人按雇佣关系分类，有人按销售队伍或应用入口分类，有人按在门店承担的职责分类。把它们全部塞进一个“人员类型”下拉框，会让权限、考核和数据范围无法解释。

本章的中心问题是：**在 Mars 业务系统中，什么叫“业务执行人员”，以及人员类型、账号来源和业务角色应该怎样组合?**

1. 人员类型不等于业务角色

一个 DSR 可以在某家店承担业务员职责，也可以只是某个经销商组织里的联系人员。玛氏员工可能以区域经理、审批人或报表使用者出现；品牌理货员也可能进入门店执行场景，却未必拥有门店维护权限。

因此至少要区分：

维度	回答
人员类型	这个人的用工/合作性质是什么
来源	这条人员记录从哪个系统或应用进入
应用账号	这个人能登录哪个应用
功能角色	这个人能使用什么功能
数据范围	这个人能查看或维护哪些门店/客户
业务关系	这个人负责哪些门店、客户或经销商

“业务执行人员”是本书的业务范围词，不是数据库里的单一枚举。纳入本书的主体，必须至少与终端覆盖、门店执行、销售管理、审批或业务数据责任之一发生关系。

2. staging 当前观察到四类 MDP 人员类型

2026-10-26 22:57 CST，`yhx_master.mdp_personnel` 中未删除人员按当前人员类型聚合为：

人员类型	行数	`on_job=1`	distinct employee
玛氏员工	33,909	6,018	33,909
DSR	15,143	2,246	15,143
品牌理货员	3,783	6	3,783
DA	922	329	922

这张表当前只观察到四类。知识库主数据文档和访问控制模型还提到 BM 外包、第三方供应商、文员、稽查等类型；它们是否存在于其它来源或历史时点，不由本次 MDP 聚合回答。

所以正文不能写“Mars 当前只有四类人员”。准确写法是：**2026-10-26 staging 的 mdp_personnel 当前观察到四类。**

另一个事实是，行数不等于当前在岗人数。上表 33,909 个玛氏员工记录中只有 6,018 条 `on_job=1`；DSR 的 15,143 行中只有 2,246 条。历史人员仍是主数据的一部分，不能从总行数直接计算组织现状。

3. MDP 不是所有应用人员的同义词

`mdp_personnel` 当前来源全部为 `MDP`，共 53,757 行。

`personnel` 表则是另一个加工/应用层人员载体。当前有 412 条未删除记录，来源包括 `cotsfa`、`dasfa` 和 `yhx`。证据中还分别记录了 `yhx` 的 9 和 3 两个统计值，但没有说明两者的分组字段和业务含义，因此本书不把它们合并成一个可解释的人数指标。

这证明 `personnel` 不是 `mdp_personnel` 的无差别镜像。它可能承载应用侧创建、修改或映射后的人员。正文不能把任意一张 `personnel` 表都写成“MDP 全量主数据”。

代码中的 `PersonnelDo` 同时声明 `appCode`、`sourceCode`、`sourceName`、`employeeTypeCode`、`onJob`、经销商字段和有效期。这说明同一条人员记录已经携带“应用”和“来源”语义，不只是姓名与工号。

4. 账号按应用分层

当前 `user_info` 未删除账号按应用和有效/冻结状态聚合为：

应用	valid=false	valid=true	frozen=true
`cotsfa`	3,359	5,564	7
`cotsfa-marketing-mg-v2`	0	7	0
`dasfa`	0	71	0
`yhx`	0	6	0

其中 `cotsfa` 仍在有效状态但有 1 个冻结账号；无效状态中也有 6 个冻结账号。

这些分布说明“账号总数”“有效账号”“可登录账号”“有业务角色账号”不是同一指标。页面或报表若只按 `COUNT(*)` 统计 `user_info`，会包含无效账号和历史账号。

`UserInfoDo` 的代码字段也把 `appCode`、`valid`、`frozen` 和 `listOfClientCodes` 分开。账号状态与客户端可见性不能合并成一个布尔值。

5. DA、DSR 与品牌理货员不是同一种角色

从当前观察可以确认：

- DSR 是 `mdp_personnel` 中的人员类型；
- DA 是 `mdp_personnel` 中的人员类型，同时在账号表中存在独立 `dasfa` 应用；
- 品牌理货员是 `mdp_personnel` 中的人员类型。

这三项说明它们至少是人员类型或来源维度。它们是否拥有门店维护权限、拜访权限或考核责任，仍需读取应用角色、数据范围和业务关系，不能由类型名称直接推出。

例如 `dasfa` 的有效账号只说明账号处于有效状态，不说明该账号能看哪些门店。人店关系中的 `source=DA` 又属于另一层关系来源，也不能与 `user_info.app_code=dasfa` 直接等价。

6. 经销商人员是关系范围，不自动成为 Mars 应用账号

知识库客户角色模型把经销商、二分商、批发商、零售商、消费者分开。用户中心另有人店关系、经销商人员映射和合作人员相关表。

一个经销商业务员可能：

- 在经销商侧系统中有身份；
- 在 Mars DA/IC 清单中作为联系人出现；

- 与某家经销商或门店建立映射；
- 能通过合作人员或第三方身份进入 Mars 应用；
- 只在报表中以执行人编码出现。

以上状态需要逐项证据。“经销商人员”进入 Mars 业务链，不代表自动拥有 Mars 账号、角色或门店数据范围。

本章只确定对象边界。具体经销商人员映射由后续“主管链与其它业务关系”章节展开。

7. 业务执行人员的判定顺序

给一个人员判断是否属于“业务执行人员”，可以按以下顺序：

1. 是否存在企业人员或合作人员记录；
2. 人员类型是否属于销售执行、门店覆盖、管理或相关支持范围；
3. 是否与门店、客户、经销商或执行记录存在业务关系；
4. 是否拥有目标应用账号；
5. 是否拥有实际功能角色和数据范围；
6. 当前是否在正确有效时点。

第 2 步命中只说明“候选执行主体”；第 3–6 步才决定其在具体项目中能做什么。

如果问题只是“历史报表中谁做过拜访”，人员可能已离职且账号无效，但仍需保留历史编码；如果问题是“谁能今天维护门店”，则必须同时验证在职、账号、角色和数据范围。

8. 三个常见失真

失真一：把人员类型当权限

人员类型能帮助筛选和解释，但不能替代应用角色。DA 类型不等于自动拥有 DA SFA 全部菜单，品牌理货员也不等于可以维护门店主数据。

失真二：把来源当身份

`sourceCode` 说明记录从哪个来源进入，不等于人员只属于该来源。一个人可能在 MDP 有人员记录，同时有多个应用账号。

失真三：把总数当现状

人员表包含离职与历史，账号表包含无效与冻结。任何“多少人可用”的结论都必须带过滤条件。

9. 本章结论与未知

本章可以支持：

1. 人员类型、来源、账号、角色和数据范围是不同维度；
2. 当前 staging `mdp_personnel` 只观察到玛氏员工、DSR、品牌理货员和 DA 四类；
3. MDP 人员表与 `personnel` 应用侧来源不是同一全量；
4. 有效账号不等于可用权限，账号还有冻结和客户端条件；
5. 经销商人员要进入 Mars 应用，必须另有账号与授权证据。

本章仍不回答：

- MDP 中四类人员的完整业务角色映射；
- `personnel` 与 `mdp_personnel` 的逐人匹配率；
- 当前所有可登录账号中，有多少拥有目标门店范围；
- 经销商人员在各业务系统中的完整映射；
- BM 外包、第三方供应商等类型在其它来源中的当前规模。

这些未知进入后续来源、权限和关系章节。

下一章讨论人员身份从 MDP 到用户中心和加工副本的逐层来源。要查询人员类型与字段，进入案头 A；要追溯系统与数据链，进入案头 B；未决的口径与维护问题集中登记在案头 D。

第 3 章 人员身份从哪里来：MDP、用户中心与加工副本

“人员来自 MDP”经常被当成一句完整答案。它没有回答：

- MDP 的哪些实体被同步；
- 同步先落到哪个原始表；
- 在用户中心经过哪些整合和映射；
- 应用人员、用户账号和综合投影是否都在同一层；
- 当前表里能否用同一个工号直接连接。

如果项目只记住“上游是 MDP”，就会把任意人员副本都当作主数据，把表行数差异当成人员增减，或者用一个看似一致的 `employee_code` 直接内连接所有层。

本章的中心问题是：**人员身份在 MDP、用户中心和应用侧之间经过哪些层，什么可以说来自上游，什么只是加工副本？**

1. 文档给出的分层链

人员与组织规则把系统分成：

- MDP 企业人员、组织和来源属性；
- 用户中心人员/组织加工副本；
- 应用账号、客户端、角色和资源授权；
- 门店侧人店关系。

数据来龙去脉进一步列出四层加工：

```
MDP 原始数据
  → DI 分层
  → 映射层
  → 期次快照 / 业务投影
```

这说明“来自 MDP”只是来源声明。真正查询时还要说明取原始、DI、映射、人员表、Mongo 投影还是门店关系。

2. staging 同步配置确认六个实体

2026-10-26 23:02 CST，`mdp_synchronous_config` 当前有 6 个有效目标：

- BM；
- DSR；
- employee；
- hierarchyG；
- hierarchyL；
- vehicle。

每个目标当前都是 `full` 抽取并保留 3 天快照。目标表都位于 `yhx_master`，名称采用 `di_olo_mdp_*`。

这证明当前配置覆盖六类数据，但不能证明：

- 每次同步都成功；
- 六个实体的字段映射完全正确；
- 后续所有人员表都立刻刷新；
- 生产具有相同配置。

同步配置表还包含接口与连通字段。本章只读取目标库、目标表、抽取类型和保留天数，不读取或记录认证凭据。

3. 原始层、加工层和应用人员层行数不同

同一时点观察到：

载体	行数
`di_otr_all_user_info`	54,154
`mdp_personnel`，未删除	53,757
`personnel`，未删除	412

三张表都可能与“人员”有关，但行数差异很大。`personnel` 不是 53,757 条 MDP 人员的简单副本，`di_otr_all_user_info` 也不等于最终应用人员表。

更值得注意的是，本次按 `employee_code` 对 `personnel` 和 `mdp_personnel` 做精确等值连接，结果为 0。

这个结果必须准确表述为：

> 当前 staging 两张表之间没有相同的 `employee_code` 字符串。

它不能直接推出：

- 两套人员完全无关；
- MDP 没有应用人员；
- `personnel` 的数据都错误；
- 应删除其中一个表。

仍有编码规则、前后空格、应用侧语义、不同人员范围或加工断点等可能，需要字段分布、主数据和运行日志继续核对。

4. `personnel` 带应用来源，不像单一主数据

`personnel` 当前来源包括 `cotsfa`、`dasfa`、`yhx`。代码 `PersonnelDo` 又声明 `appCode`、`sourceCode`、`sourceName`、人员类型和有效期。

这说明应用侧人员层可能承载各应用自己的维护和消费范围。它可以不是“全 Mars 企业人员”，而是用户中心为应用构建的人员记录。

`mdp_personnel` 当前来源全为 `MDP`，更接近上游人员加工结果。但“更接近”仍不等于它就是原始主数据，因为它是用户中心中的表，不是 MDP 原表。

本书使用以下称呼：

- **MDP 原始数据**：同步落地区；
- **用户中心加工层**：DI、映射和人员整合；
- **应用人员层**：按应用维护/消费的人员；
- **应用账号层**：`user_info`；
- **业务关系层**：人店关系、主管链、客户/经销商人员关系。

5. 一个工号不是一个跨表万能连接

当前精确连接为 0 的观察说明，跨表对账不能只写：

```
FROM personnel p
JOIN mdp_personnel m ON p.employee_code = m.employee_code
```

就假定结果应覆盖全部人员。项目必须先回答：

1. 两张表的 `employee_code` 生成规则是否相同；
2. 是否允许前后空格、大小写或格式转换；
3. `personnel` 是否只包含应用自建人员；
4. 是否需要通过 `uid`、ad number、third-party identity 或映射表连接；

5. 两边是否在同一时间范围和来源范围。

在连接规则未确认前，用名称或手机号强行匹配会引入更大风险。人员身份对账必须有正式业务键和脱敏比对方案。

6. 同一人的不同“当前状态”

不同层有不同状态：

- 人员层有 `on_job`、入离职日期和有效期；
- 账号层有 `valid`、`frozen` 和客户端列表；
- 人店关系有 `state` 与有效起止时间；
- 组织关系有期次主管链或组织节点。

“人员当前有效”“账号可登录”“关系当前有效”必须分别判断。

一个人离职后，其账号可能仍保留历史；一次关系到未来才生效；一个授权组织可以被回收，而人员主数据仍存在。所有层都用“当前”一词时，实际可能指不同时间点。

7. 来源链的验收方法

验证一条人员身份链，应至少记录：

证据	要证明什么
同步配置	哪个场景同步到哪张原始表
原始表快照	上游是否到达
加工映射	字段怎样转换、过滤和合并
人员表	加工结果及有效期
账号表	应用身份和状态
组织/关系	业务责任与范围
下游投影	查询和报表何时看到结果

只验证最后一个页面，无法判断数据来自哪一层；只验证同步成功，也无法判断应用人员身份已经建立。

8. 反例

反例一：把 `di_otr_all_user_info` 当生产人员清单

它是加工层表，当前行数也有自身范围和快照语义。直接当“当前人员总数”会跳过人员和账号状态。

反例二：把 `personnel` 当 MDP 全量

当前 `personnel` 只有 412 条未删除记录且来源是应用侧，而 `mdp_personnel` 有 53,757 条，二者不能互换。

反例三：零交集就判错

精确工号连接为 0 是重要异常信号，但根因仍需继续查编码和来源。把异常立即写成结论会掩盖真正链路。

9. 本章结论与未知

本章可以支持：

1. MDP 同步配置当前覆盖 BM、DSR、employee、hierarchyG、hierarchyL、vehicle 六类；
2. MDP 原始、用户中心加工、应用人员和账号层是不同层；
3. 当前三层人员表行数明显不同；
4. 当前 `personnel` 与 `mdp_personnel` 按 `employee_code` 精确连接为 0；
5. 来源链必须逐层验证，不能只看一个页面。

本章仍不回答：

- `personnel` 与 `mdp_personnel` 的真实连接规则；
- 54,154 与 53,757 行差 397 行的业务原因；
- 六类 MDP 数据的逐字段映射完整性；
- 同步任务最近成功时间与失败重试；
- 生产环境是否具有相同链路和差异。

下一章从此转入组织节点、主管链和玛氏地理，说明人员归属、汇报范围和区域数据为什么不能共用一个“组织”答案。对象字段见案头 A，系统数据链见案头 B，尚未闭合的映射问题见案头 D。

第 4 章 组织、主管链与玛氏地理

“这个用户属于哪个组织”“谁是他的主管”“他能看哪个区域”是三个不同问题，却经常被同一个 `org` 参数或姓名覆盖。

组织树决定节点位置与层级；主管链决定向上汇报和审批人解析；玛氏地理决定资源范围。人员可以属于一个组织、获得额外授权组织，并在人员记录上带玛氏地理字段，但这些关系不能压成一个部门名称。

本章的中心问题是：**人员怎样进入组织、主管链和地理范围，三者分别在什么时候使用？**

1. 组织容器和组织节点不是同一层

用户中心的 `organization` 当前只有 7 行，但 `organization_content` 在 2026-10-26 未删除节点为 8,794 个，最大层级为 9。

组织容器声明需要哪些资源分类与权限维度；组织内容是具体树节点，包含：

- `parent_uid`；
- `level`；
- `uid_path`；
- `state`；
- `version`；
- 有效期。

如果报表统计“组织数”时读 `organization`，得到 7；如果读 `organization_content`，得到 8,794。两个数字都真实，但回答不同问题。

2. 自身组织和授权组织分开维护

当前 staging：

- `user_org` 未删除 8,751 行；
- `grant_user_org` 未删除 482 行。

`user_org` 表示用户归属的自身组织；`grant_user_org` 表示额外授予的组织范围。

代码 `UserOrganizationBizServiceImpl.getUserOrgAndGrantMixOrgHierarchy` 分别读取自身组织和授权组织，再把组织内容 UID 合并到同一个层级结果。

因此，只查 `user_org` 会漏掉授权范围，只查 `grant_user_org` 会漏掉基本归属。最终数据权限还要继续连接组织节点、资源分类和业务范围。

3. 主管链不是组织树当前值

人员主数据和组织规则将主管链作为期次快照处理。staging 当前观察到的最新期次表为：

```
yhx_master.di_otr_user_org_supervisor_2026p10
```

该表 8,921 行，对应 8,921 个 distinct `user_code`，`build_time` 为 2026-10-03 20:10:03。

表字段包括：

- `user_code`；
- `supervisor_code` / `supervisor_name`；
- `level`；
- 组织编码和名称层级；
- `build_time`。

这使审批人解析可以带期次：查询 2026P10 的审批链时，不应直接使用今天组织树上的最新父节点。

4. 门店主管投影是另一个表面

门店侧还有 `store_ops.store_emp_supervisor`。当前有 1,693,161 行，但最新更新时间停在 2025-01-24 14:13:58。

它与 2026P10 人员主管链期次表不是同一张表，也不能因为名字都包含 supervisor 就默认同步。

当前至少存在三类主管/组织表面：

表面	回答	当前观察
用户中心组织树	用户属于哪个组织节点	8,794 节点、9 层
期次主管链	某期用户向上主管链	2026P10，8,921 用户
门店主管投影	门店侧业务员/负责人对应主管	1,693,161 行，最新 2025-01-24

做审批、门店督导和报表时，要先确认目标入口读取哪一层。

5. 玛氏地理不是组织层级

人员表声明两组地理信息：

- 国家行政地理：`geo_*`、省市区县等；
- 玛氏销售地理：`mars_geo_code`、`mars_region_*`、`mars_province_*`、`mars_city_group_*`、`mars_city_*`。

组织树回答“在组织里归谁”；玛氏地理回答“覆盖哪个销售区域”。同一个组织节点可能绑定多个资源，组织内容与资源节点通过 `organization_content_category_node` 关联。

因此，页面上的“区域”可能是：

- 国家行政区；
- 玛氏销售区域；
- 组织节点名称；
- 用户自身组织；
- 授权组织范围；
- 人店关系或线路。

必须看到来源字段和用途，才能知道它代表哪一种区域。

6. 查询一个用户的范围

可执行顺序：

1. 用 `employee_code` / `user_code` 定位用户与人员；
2. 读取自身组织 `user_org`；
3. 读取授权组织 `grant_user_org`；
4. 合并组织内容节点；
5. 通过节点绑定资源解析玛氏地理、渠道、组织层级等范围；
6. 按业务应用、角色和有效期过滤；
7. 如果涉及审批，另按目标期次读取主管链表；
8. 如果涉及门店督导，检查门店主管投影的时点。

不能在第 2 步看到自身组织后就宣布范围完成。

7. 三个反例

反例一：组织容器数当组织节点数

`organization=7` 与 `organization_content=8,794` 不是同一层计数。

反例二：自身组织当完整数据范围

忽略 `grant_user_org` 会让授权用户看不到应得数据。

反例三：最新主管链当历史主管链

人员组织可能调整，但历史期审批必须保留当时的期次主管关系。按当前组织父节点重算会改变历史审批解释。

8. 本章结论与未知

本章可以支持：

1. 组织容器与组织内容是不同层；
2. 自身组织与授权组织分开维护，查询时合并；
3. 主管链按期次保存，最新观察到 2026P10；
4. 门店主管投影与期次主管链是不同表面；
5. 国家行政地理、玛氏销售地理和组织节点不是同一范围。

本章仍不回答：

- 门店主管投影最新时间停在 2025 年的原因；
- 2026P10 主管链是否已覆盖所有业务人员和业务入口；
- `grant_user_org` 的审批与到期回收机制；
- 组织节点与每类资源的实际映射覆盖率；
- 各应用最终数据权限是否都在目标时点消费了最新组织。

下一章进入用户、账号与第三方身份，继续回答“组织里存在这个人”之后，他怎样通过某个认证入口进入应用。人员与组织字段见案头 A，认证与数据链见案头 B，主管链缺口和维护事项见案头 D。

第 5 章 用户、账号、登录与第三方身份

一个人的业务身份可能同时涉及工号、应用账号、AD 号、企微 userid 和 OKTA 身份。“能登录”到底是在验证哪一个?

用户中心并不把人员、账号、第三方身份和登录入口保存在一条记录里。它把这些对象分开，再由登录入口按 redirect 定位应用与客户端。认证成功之后，功能权限和数据范围仍未开始。

本章的中心问题是：**用户、账号、第三方身份和登录方式之间的关系是什么，哪些状态共同决定认证结果?**

1. 用户不是一个第三方账号

`user_info` 表示某个人在某应用下的用户账号，包含：

- `employee_code`；
- `app_code`；
- `account`；
- `valid`；
- `frozen`；
- `idp_source`；
- `list_of_client_codes`；
- `last_login_date_time`。

`user_thirdparty_info` 则表示用户与外部身份之间的映射：

- `employee_code`；
- `third_party_account_number`；
- `third_part`；
- `app_code`；
- `state`；
- 有效期。

同一个员工可以有多个第三方身份，所以在 `user_thirdparty_info` 中不能按员工数等于身份行数。

2. 当前身份以 OKTA 和企微为主

2026-10-26 23:13 CST，未删除第三方身份共 23,839 行，对应：

- 19,168 个 distinct employee；
- 23,644 个 distinct 第三方账号值。

按平台与应用聚合：

平台	应用	身份行	distinct employee
EE_WECHAT	cotsfa	18,784	18,782
OKTA	cotsfa	4,781	4,781
EE_WECHAT_MARS_CN	cotsfa	168	167
EE_WECHAT	dasfa	68	68
OKTA	dasfa	27	27
OKTA	cotsfa-marketing-mg-v2	7	7
OKTA	yhx	4	4

这说明认证身份要按“平台 × 应用”解释。同一个人在企业微信和 OKTA 下可能有不同外部账号。

3. 客户端定义认证通道

当前 `client` 表在 `is_deleted=0` 条件下按 `thirdpart_code` 聚合：

认证通道	客户端行	启用客户端
OKTA	14	14
EE_WECHAT	3	3

客户端不是应用本身。一个应用可以注册多个客户端，例如后台、企微端或协访端。登录入口先通过 `redirect` 定位唯一的应用与客户端，再使用对应认证方式。

如果 redirect 匹配不到唯一客户端，后续用户账号状态再好也无法完成正确登录。

4. 代码提供四种认证入口

当前 `AuthenticationApi` 存在：

端点	身份输入	文档与代码语义
`/auth/check/account_user`	`username` + app	按账号定位用户
`/auth/check/okta_user`	`adNumber` + app	按 AD 号定位用户
`/auth/check/wechat_user`	企微身份 + app	按企微用户定位
`/auth/check/mc_wechat_user`	Mars China 企微身份 + app	独立企微通道

账号入口代码注释明确“没有密码校验”。所以 `user_info.password` 列存在，不代表当前账号入口在执行本地密码校验。认证职责可能由外部身份提供方和上游协议承担。

项目不能因为代码里存在 password 列就设计一套本地密码流程，也不能因为账号入口叫 `account_user` 就认为它验证用户名密码。

5. 认证结果要逐项检查

以 OKTA 入口为例，当前代码会检查：

1. redirect 能否唯一定位应用/客户端；
2. AD 号在该应用中能否找到唯一用户；
3. 用户 `frozen` 是否为真；
4. 用户 `valid` 是否为真；
5. 关联人员是否存在且唯一；
6. 人员 `on_job` 是否为真。

这里出现了用户账号与人员两条状态链。账号有效不等于人员在岗；人员仍在职也不等于已经建立目标应用的账号。

企微入口还会使用企业微信身份映射和目标应用客户端配置。具体分支应以对应代码为准，不能把 OKTA 规则直接套成所有认证方式。

6. `idp_source` 和最近登录当前不能当认证事实

当前 `user_info` 未删除账号中：

- `idp_source` 全部为空；
- `list_of_client_codes` 非空 164 行；
- `last_login_date_time` 非空 0 行。

因此，当前 staging 不能通过 `idp_source` 判断用户来自 OKTA 还是企微；真正的平台映射在 `user_thirdparty_info.third_part`。

也不能用最近登录为空判断用户从未登录。该字段当前未记录，说明的是证据不足，不是业务行为不存在。

`user_thirdparty_info` 还有 `state` 字段，但当前 `re-yhx-idp` 的 OKTA/企微查询只按应用、工号、`third_part` 和账号值连接，没有按 `state` 过滤。因此不能把“存在 state 字段”直接写成“认证层已经执行启用状态门禁”；若业务要求停用绑定必须阻断登录，应另做实现与验收。

7. 第三方身份行数不是覆盖人数

19,168 个 distinct employee 有 23,839 个身份行，说明一部分人拥有多个身份。

但它也不能直接回答：

- 有多少人可以登录；
- 有多少人持有有效客户端；
- 有多少人拥有目标应用角色；
- 多少人能访问目标门店。

要回答这些，还必须连接 `user_info.valid/frozen`、`client`、`role_client`、`role_menu`、组织和资源范围。

8. 认证、授权与数据权限

层	回答	当前对象
认证	这个人是谁、能否证明身份	第三方身份、认证入口
账号	这个应用是否接受该用户	`user_info`
功能授权	能打开哪些客户端和菜单	role/client/menu
数据权限	能看或维护哪些门店/区域	组织、资源、授权、人店关系

登录成功最多证明前三层中的部分条件。它不能证明数据范围正确。

9. 本章结论与未知

本章可以支持：

1. 用户、账号和第三方身份分层；
2. 当前第三方身份以企微和 OKTA 为主；
3. 一个员工可有多个身份；
4. 认证入口按 redirect 定位应用与客户端；
5. 账号入口当前没有本地密码校验；
6. `idp_source` 和最近登录字段当前不能用于运行判断。

本章仍不回答：

- 23,839 个身份中多少能实际登录；
- 19,168 个员工与 9,000 余账号的完整映射；
- 企业微信同步任务当前是否仍按历史配置运行；
- 所有客户端 redirect 配置是否唯一；
- 登录成功后目标用户的数据范围是否正确。

下一章顺着客户端进入应用、角色和菜单，拆解功能权限链。账号字段见案头 A，认证与系统链见案头 B，项目开通和正负验收见案头 C。

第 6 章 应用、客户端、角色与菜单权限

“用户登录了”与“用户看到了菜单”之间，还隔着应用、客户端、角色和菜单四层。一个应用可以有多个客户端；一个角色可以绑定多个客户端；同一菜单也可能由多个角色授予。

如果把角色当成应用，把菜单当角色，或用客户端 code 代替应用 code，项目就会出现“账号有、页面没有”“菜单有、数据没有”“后台能看、企微端不能进”等不同故障。

本章的中心问题是：**功能权限从用户到菜单要经过哪条关系链，各层当前有哪些独立状态？**

1. 应用和客户端不是同一层

当前 `app_content` 有 5 个启用应用：

应用	名称
cotsfa	COT SFA
cotsfa-marketing-mg-v2	营销中心
dasfa	DA SFA
lyktest1	云开测试应2A
yhx	用户中心A

应用是接入方，不是具体界面。当前未删除 `client` 有 17 行且 state 均为 1：

应用	客户端行	认证通道
cotsfa	13	OKTA 11、EE_WECHAT 2
cotsfa-marketing-mg-v2	1	OKTA
dasfa	2	OKTA 1、EE_WECHAT 1
yhx	1	OKTA

这里有 5 个应用，却有 17 个客户端，说明“应用授权”和“客户端入口授权”不是同一层。一个用户在同一个应用下也可能只能进入特定客户端。

2. 角色属于应用，不是人员属性

当前角色按应用和状态：

应用	状态	角色行
cotsfa	0	3
cotsfa	1	33
cotsfa-marketing-mg-v2	1	4
dasfa	1	2
yhx	1	2

44 = 41 个启用 + 3 个停用。角色属于应用，不是人员属性。

角色不只保存名称。代码实体还包含应用、角色 code、超级管理员限制、是否可授销售组织、是否可授配置测试和状态等字段。

3. 用户角色把账号连接到角色

当前 `user_role` 关系：

- 9,824 行；
- 8,891 个 distinct user；
- 41 个 distinct role_code。

这一层把用户指向角色，不直接指向菜单。用户能否打开某个客户端，还要通过 `role_client`；能打开哪些功能，再通过 `role_menu`。

9,824 条关系不能直接解释为 9,824 个可登录用户。关系可能覆盖不同应用、不同生效时间和不同账号状态。

4. 角色客户端决定功能入口

当前 `role_client`：

- 49 行；
- 涉及 31 个 role_code；
- 涉及 6 个 client_code。

17 个未删除客户端中，只有 6 个当前有角色绑定。这不等于其它客户端都无业务意义；可能由前端硬编码路由、尚未完成配置或客户尚未开通。文档已把菜单与前端路由同步列为开放问题。

5. 角色菜单定义功能集合

当前 `role_menu`：

- 604 行；
- 涉及 31 个 role_code；
- 涉及 157 个 menu_uid。

当前 `menu` 共 210 条启用记录，集中于：

- COT+：38；
- 公海门店后台：76；
- 营销中心 campaign-page：30；
- DA 后台：29；
- 用户中心后台：26。

其余多个客户端只有一条菜单记录或缺菜单。菜单存在不等于业务用户可见，还要继续看角色绑定和客户端。

6. 完整功能权限链

当前文档确定的判定链：

```
User
  → UserRole
  → Role
  ├─ RoleClient → Client
  └─ RoleMenu → Menu
```

其中 role code 与 client code 是关系键，而不是层级目录。

功能权限链回答：

- 用户属于哪些应用；
- 持有哪些角色；
- 角色绑定哪些客户端；
- 每个角色可访问哪些菜单；
- 客户端当前是否启用。

它不回答：

- 用户能看哪些门店；
- 能维护哪些客户；
- 是否有数据导出权限；
- 动作级权限是否完成。

数据范围继续由组织和资源决定。

7. 权限验收要逐层做正负样本

层	正例	反例
应用	用户属于目标应用	只有别的应用账号
客户端	账号可进入目标客户端	应用有效、客户端未绑定
角色	目标角色已授予	有账号无角色
角色客户端	角色绑定目标客户端	角色存在但不适用于该端
角色菜单	菜单授予角色	角色绑定客户端但无菜单
菜单路由	页面加载目标菜单	前端路由缺失或硬编码
用户可见	正样本菜单可见	越权菜单不可见

只验证前一层不能证明后一层成功。

8. 当前数据里的两个风险

风险一：对象存在不等于开通完成

17 个客户端、41 个启用角色、210 个菜单均存在，却只有 31 个角色有客户端和菜单关系。项目开人仍必须用目标真人实际登录并检查正负样本。

风险二：异常客户端 code

当前菜单聚合中出现 `yhx / client_code=213213` 的一条记录。它不应被直接解释为有效业务端。先登记为疑似测试或脏值，等业务和配置 Owner 确认。

9. 本章结论与未知

本章可以支持：

1. 应用、客户端、角色、菜单是不同对象；
2. 当前有 5 个应用、17 个未删除客户端、41 个启用角色和 210 个启用菜单；
3. 用户角色、角色客户端、角色菜单分别有 9,824/49/604 条当前关系；
4. 功能权限必须逐层验收；
5. 功能权限与数据权限是不同范围。

本章仍不回答：

- 每个客户端的真实可登录用户数；
- 菜单与前端路由如何同步；
- `yhx/213213` 的来源；
- 无客户端或菜单绑定的角色是否应保留；
- 应用角色与数据范围在生产的一致性。

下一章从功能入口转向自身组织、授权组织、资源和门店范围，说明“能打开页面”为什么仍不等于“能看对数据”。字段与关系见案头 A，权限系统链见案头 B，开通和越权测试见案头 C。

第 7 章 数据权限：组织、资源与门店范围

用户能看到菜单，不表示能看到目标门店。功能权限回答“能打开哪个页面”，数据权限回答“页面里哪些数据能被查询、维护或导出”。

门店数据范围通常不是单个组织 ID，而是“自身组织 ∪ 授权组织”展开后得到的资源集合，再与 segment、玛氏地理、覆盖模式预设和具体入口规则相交。

本章的中心问题是：**组织与资源怎样变成用户可见、可维护的门店范围？**

1. 从组织关系开始

当前 staging：

关系	当前行	distinct user
`user_org`	8,751	8,627
`grant_user_org`	482	205

自身组织是基础归属，授权组织是额外范围。代码 `OpenStoreValidator` 在请求用户资源时传入：

```
org_types=self,grant
```

这直接证明公海门店数据权限会同时考虑自身和授权组织。

2. 组织节点通过资源分类展开

当前 `organization_content_category_node` 有 115,455 条未删除绑定，涉及：

- 8,874 个组织节点；
- 7 个资源分类；
- 663 个资源节点。

按分类：

资源分类	绑定行	组织节点	资源节点
mars_channel	87,698	8,770	14
mars_hierarchyG	9,340	8,777	456
mars_position	9,337	8,759	23
mars_segment	8,858	8,858	2
city_headquarters	217	102	167
mars_hierarchyL	4	4	2
mars_test	1	1	1

渠道资源只有 14 个节点，却产生最多绑定；玛氏地理 G 有 456 个节点。不同资源分类的节点规模和绑定方式不同。

3. 当前资源节点规模

`resource_content` 当前节点：

资源分类	节点
mars_hierarchyL	43,360
city_headquarters	15,578
mars_hierarchyG	487
mars_channel	145
mars_position	22
mars_test	3
mars_segment	2

组织绑定涉及的资源节点数 663 小于资源表总量，说明绑定只使用资源全集的一部分，或不同分类节点未全部被组织引用。不能把“资源存在”当成“已授权”。

4. 公海门店的数据权限链

当前代码 `OpenStoreValidator` 的处理顺序：

```
用户标签 /open/user_label
  → 读取 mars_segment
  → /open/user/org/belongWithResource
  → org_types=self,grant
  → 读取 mars_segment + mars_hierarchyG 资源
  → 展开 region / province / city cluster / city
```

对 MW 与 IC：

- 先读取用户 `mars_segment`；
- 再分别展开自身和授权组织的资源；
- 按 segment 过滤组织资源；
- 最终生成玛氏区域、省、城市群和城市集合。

这说明功能页面只提供了入口，实际门店 SQL 仍需使用这些资源值过滤。

5. 门店范围模型

门店中心数据权限权威文档给出：

```
可查询和维护门店范围
= 覆盖模式预设 ∩ segment ∩ 授权玛氏地理
```

同一用户可能要同时支持：

- 糖巧和冰淇淋 segment；
- 自身组织与授权组织；
- 区域内和双覆盖门店；
- 菜单预设的覆盖模式。

“菜单可见”“组织可见”“门店可维护”是三步判断。

6. 应用级组织授权

当前 `grant_organization_content_app` 只有：

- cotsfa：8 行 / 8 节点；
- dasfa：1 行 / 1 节点。

应用级组织授权表示某应用可见的组织子集。它不是用户授权组织，也不是客户端菜单权限。

一个应用能看到组织节点，不代表每个用户都能看到；一个用户有自身组织，也不代表目标应用已获得该组织。

7. 查询与维护可能不同

文档区分：

- 门店信息维护按组织地理控制；
- 人员信息维护按“自身及以下 + 授权及以下”控制。

同一页面内，不同字段可能使用不同范围。用户能看到门店，不代表能改所有字段；能改门店基础信息，不代表能改人店关系。

数据权限验收必须声明“入口 + 动作 + 资源类型”。

8. 正负样本矩阵

编号	场景	预期
DP-01	自身组织内、玛氏地理内门店	可查询
DP-02	授权组织内、玛氏地理内门店	可查询
DP-03	自身组织但玛氏地理外门店	不可见
DP-04	玛氏地理内但 segment 不匹配门店	不可见
DP-05	功能菜单有权限但组织无资源	不返回越权门店
DP-06	组织可见但动作无权限	可查不可改
DP-07	双覆盖门店	按 segment 和覆盖预设正确拆分
DP-08	授权组织与自身组织重叠	去重，不重复行

9. 本章结论与未知

本章可以支持：

1. 数据权限由自身组织、授权组织、资源和业务范围共同决定；
2. 当前组织资源绑定 115,455 条，涉及 7 个资源分类；
3. 当前自身组织覆盖 8,627 个 user，授权组织覆盖 205 个 user；
4. 公海门店代码显式请求 self + grant 资源；
5. 功能权限和数据权限必须分开验收。

本章仍不回答：

- 205 个授权组织的来源与回收流程；
- 663 个已绑定资源节点与全部资源节点的完整覆盖率；
- 每个菜单实际使用的门店预设；
- 查询、维护、导出动作的配置差异；
- 各应用 SQL 是否都正确使用同一范围。

下一章把数据范围落到人店关系：一条当前有效责任必须同时说明 store_code、duty_type_code、来源、状态和有效期。字段模型见案头 A，关系写入与消费链见案头 B，变更和验收矩阵见案头 C。

第 8 章 人店关系：业务员、门店负责人、来源与生效时间

“这家店由谁负责”在页面上可能只是一个姓名，在系统中却是一条带状态和有效期的关系。同一门店可以同时有业务员和负责人，也可以有 COT、IC、DA 多个来源。

如果项目只按 `store_code + emp_code` 去重，会把不同职责、不同来源或未来关系合并；只按当前姓名查询，又会丢失历史。

本章的中心问题是：**一条有效的人店关系由哪些键和状态共同定义？**

1. 最小关系键

人店关系的最小事实不是：

```
store_code + emp_code
```

而是至少包含：

```
store_code
+ emp_code
+ duty_type_code
+ source
+ state
+ effective_start_timestamp / effective_end_timestamp
```

`duty_type_code=L` 是业务员，`F` 是门店负责人。同店同人可同时兼任 L/F。`source` 区分 COT、IC、DA 等入口。

2. 当前有效关系分布

2026-10-26 23:28 CST，以 `state=1` 和当前时间窗过滤：

source	duty	关系行	distinct 门店	distinct 人员
COT	F	1,739,606	1,739,450	7,336
COT	L	1,757,728	1,757,572	8,021
DA	F	115	115	19
DA	L	115	115	19
IC	F	4,921	4,921	42
IC	L	4,921	4,921	42

在 COT 范围内，F 与 L 的门店数和人员数接近，但不代表每店各一条。页面和 SQL 必须按职责聚合，不能按 `COUNT(*)` 直接等同门店数。

3. 状态与有效期必须同时判断

早先的全状态聚合显示，当前时间窗内除了 state=1，还存在审批中 state=2 和少量 state=0/4。

所以“时间窗内”不等于“已生效”：

```
current effective
= state=1
  AND start <= now
  AND end > now
  AND not deleted
```

如果查询将来生效的关系，还要单独保留未来行；如果查历史，则不能只看当前。

4. 关系采用拉链更新

当前 `re-sops` 代码：

- `StoreEmpDo` 对应 `store_emp`；
- upsert/import 计算生效日期；
- 修改时关闭旧区间并写入新关系；
- 新增关系从当前或配置的生效时点开始。

这保证历史负责人可追溯。页面覆盖姓名不是事实模型。

5. 人店关系与主管链不同

人店关系回答：

```
谁负责这家门店
```

主管链回答：

```
这个人向谁汇报
```

门店侧还有 `store_emp_supervisor` 投影，当前 1,693,161 行，但最新更新时间停在 2025-01-24；最新用户中心期次主管链是 2026P10。

因此：

- 门店有负责人，不等于主管链新鲜；
- 用户中心主管链有数据，不等于门店投影已刷新；
- 审批人解析和门店督导可能读取不同表面。

6. 人店关系与线路不同

人店关系表达责任，线路表达拜访计划与门店组合。门店可以被负责人维护，但未进入某条线路；也可以被拜访，但历史负责人已经改变。

涉及拜访覆盖时，必须同时检查：

- `store_emp` 当前关系；
- 线店关系；
- 拜访计划；
- 实际 Visit。

7. 一个门店关系查询模板

```
SELECT store_code, emp_code, duty_type_code, source,
       state, effective_start_timestamp, effective_end_timestamp
FROM store_ops.store_emp
WHERE store_code IN (:store_codes)
  AND is_deleted = 0;
```

先保留所有状态，再在应用层按目标时点判断“当前有效”。这样能解释审批中、未来和历史，而不是被一个 WHERE 条件抹掉。

8. 项目验收矩阵

编号	场景
SP-01	业务员和负责人为同一人
SP-02	业务员和负责人为不同人
SP-03	同店同职责不同 source
SP-04	审批中关系不应当有效
SP-05	未来关系在生效前不可见为当前
SP-06	修改关系关闭旧拉链、开启新拉链
SP-07	批量导入与页面写入使用同一 SoT
SP-08	综合信息和报表在承诺时点刷新
SP-09	离职后关系关闭、权限回收、历史保留
SP-10	主管链与负责人/业务员匹配

9. 本章结论与未知

本章可以支持：

1. 人店关系使用 store + person + duty + source + state + effective time；
2. 当前 COT/IC/DA 都有有效关系；
3. 当前时间窗内也存在审批中关系；
4. 关系写入采用拉链而非页面字段覆盖；
5. 人店关系、线路和主管链是不同关系。

本章仍不回答：

- 当前所有关系的完整业务唯一性；
- IC/DA/COT 多 source 门店的投影覆盖差异；
- 门店主管链停在 2025 年的原因；
- 人店关系变更后在所有下游的实际刷新时延；

- 生产与 staging 的关系分布是否一致。

下一章比较主管链、合作方、经销商人员和项目成员，继续划线说明不同业务关系为何不能互相替代。关系字段见案头 A，多系统映射见案头 B，对账和访问工作台见案头 C。

第 9 章 主管链与其它业务关系

一个“人员关系”可能指主管汇报、门店责任、客户联系人、经销商业务员、项目成员或组织合作方。它们都能把一个工号连接到另一个业务对象，却不能互相替代。

当审批人找不到、客户看不到负责人、门店报表按主管汇总为空时，问题可能落在不同关系表。先确认关系类型，才能选对证据。

本章的中心问题是：**人员的业务关系有哪些主要类型，各自用什么键和状态回答什么？**

1. 主管链

期次主管链回答“某期这个人向谁汇报”。

当前最新观察：

- `di_otr_user_org_supervisor_2026p10` 8,921 行；
- 8,921 个 distinct user；
- `build_time` 为 2026-10-03 20:10:03。

主管链用于审批人解析、主管查看和逐级汇总。查询时必须带期次，不能用今天组织父节点改写历史。

2. 门店主管投影

门店侧 `store_emp_supervisor` 当前 1,693,161 行，但最新更新时间停在 2025-01-24。

它是门店×人员的主管投影，不等于 2026P10 用户主管链，也不等于当前人店关系。

涉及门店督导或审批的项目必须单独验证：

- 目标门店是否在投影中；
- 当前业务员/负责人是否匹配；
- 投影是否在承诺时点刷新；
- 审批端实际读取哪张表。

3. 客户与合作方人员

当前用户中心：

- `org_cooperate_user` 9,217 行；
- `cooperate_user_mapping` 339 行。

两者规模不同。文档已把两表分工登记为开放问题，因此不能把它们合并成“合作方人员全集”。

合作方人员可能来自经销商、服务商或其他外部组织。进入组织体系不等于拥有 Mars 应用账号，也不等于拥有门店数据权限。

4. 经销商人员关系

2026-10-27 staging：

载体	行数	distinct 经销商	空人员编码
`store_mg_xg.distributor_emp`	50	19	0
`store_mg_xg.distributor_emp_da`	75	3	0
`yyx_da_master.distributor_emp_da`	99	12	0
`yyx_icsfa_master.distributor_emp`	50	19	0

字段只有经销商 ID、名称和人员编码。它是业务映射，不是完整人员主数据，也没有该表的拉链/审批证据。

同名表在多个库均有数据，不能按表名合并成一条关系。

5. 项目成员

项目成员由项目仓 `members.yaml` 管理，回答“此人在项目中承担什么责任”。

项目成员不是业务账号、角色或门店负责人。只在 Git/文档协作的人不需要 Mars 业务账号；需要进入业务系统的人必须另走用户、客户端、角色、组织和数据范围流程。

AI Agent 当前也没有独立业务账号或项目成员对象。

6. 关系类型对照

关系	主键或连接键	回答	不能替代
期次主管链	user + period	向谁汇报	当前组织、项目成员
门店主管投影	store + emp	门店督导链	用户中心期次主管链
人店关系	store + emp + duty + source + state + time	谁负责门店	线路、拜访
客户/合作方关系	人员 + 合作方/组织	外部人员如何进入组织或客户链	应用权限
经销商人员	dt_id + emp_code	经销商与 Mars 人员的映射	人员主数据
项目成员	project + person + role	项目责任	业务系统身份

7. 跨关系排障顺序

遇到“这个人为什么看不到/审批不到/报表没有”：

1. 问题属于哪种关系；
2. 关系是否存在；
3. 业务键是否一致；
4. 状态与生效时间；
5. 目标系统读取哪张关系或投影；
6. 权限是否另行授予；
7. 刷新与缓存时点。

不要从“加一个角色”或“改门店负责人”开始，除非症状已经定位到那一层。

8. 本章结论与未知

本章可以支持：

1. 主管链、门店主管投影、人店关系、合作方、经销商和项目成员不同；
2. 2026P10 主管链有 8,921 个用户；
3. 门店主管投影最新时间停在 2025-01-24；
4. 当前合作方组织和映射表规模不同；
5. 经销商人员映射存在多库多表副本。

本章仍不回答：

- `org_cooperate_user` 与 `cooperate_user_mapping` 的最终分工；
- 多库经销商人员表的主副本和更新责任；
- 门店主管投影停滞的业务影响范围；
- 客户联系人与组织合作方的统一关系模型；
- 各关系在生产环境的当前覆盖。

下一章沿着这些关系进入拜访、审批、ttf 和报表，检查人员事实怎样经过投影变成业务结果。关系模型见案头 A，消费链见案头 B，项目验收样本见案头 C。

第 10 章 人员怎样进入拜访、审批、ttf 和报表

页面已经显示新业务员，拜访计划却仍派给旧人；账号已冻结，审批任务仍能解析到该组织；人店关系已生效，报表仍按旧负责人汇总。这些现象容易被归为“人员主数据没同步”，实际断点可能在任何一层：关系事实源（SoT）、延迟消息、综合信息投影、线路、计划、组织主管链、流程任务或报表水线。

人员不是一个被所有系统实时读取的中心字段。不同业务面通过不同键、不同副本和不同时点消费人员及关系。判断“人员变更已经完成”，必须逐个消费目的验收。

本章的中心问题是：**一个人怎样从身份和组织关系进入拜访、审批、ttf 与报表，哪些环节可以共用证据，哪些必须分别回读?**

1. 四个业务面，四种问题

业务面	核心问题	首要人员键	首要不变量
拜访	谁负责哪家店、哪天该访	user/emp + store + date	人店关系、线店关系、计划
审批	谁应处理哪个组织任务	发起人、主管组织、任务 assignee	当前组织、代理授权、流程版本
ttf	门店采用单人还是双人模式、如何折算	门店关系与人员模式	有效人店关系、覆盖模式
报表	按谁、哪个组织或哪家店汇总	user/emp、组织、store	维度快照、期次、投影水线

它们共享部分人员身份，但没有任何单一字段能同时证明四者正确。账号可以登录，不代表计划有该人；门店有负责人，不代表报表已刷新；审批组织存在，也不代表任务执行权限成立。

2. 共同起点不是账号，而是业务关系

一个用户进入业务执行链，通常至少涉及：

```
企业人员 / 第三方执行人员
  → 应用账号
  → 组织与主管链
  → 角色与数据范围
  → 门店 / 线路 / 客户关系
  → 计划、审批、采集或报表结果
```

这些层不能相互替代：

- 账号存在，不证明人员在岗；
- 有角色，不证明能看到目标门店；
- 有门店人店关系，不证明已进入线路或计划；
- 有主管关系，不证明当前审批任务已指到该组织；
- 有拜访记录，不自动证明当时关系当前仍有效。

本章只解释人员作为执行主体和维度如何被消费；门店、产品和分销的计算口径回到各自主责书。

3. 拜访：先分清三种“关系”

拜访链至少有三类关系：

关系	回答	主要载体
人店关系	谁负责这家店、承担什么职责	`store_ops.store_emp`
线店关系	这家店属于哪条线路	`store_visit_planning.store_route_binding`
拜访计划	某人某天应访哪些店	`store_visit_planning.store_planning`

当前 staging 观察中：

- 当前有效的线店绑定有 1,336,100 行，涉及 1,299,776 家门店和 72,799 条线路；
- `store_planning` 只有 10 行，涉及 2 个 user_code 和 10 家门店；
- 计划日期从 2026-07-30 延伸到 2035-07-30，最新更新为 2026-10-27 06:30:01。

这组数不能互相替代。线店关系广，不代表计划已被生成；计划行存在，也不代表覆盖了全部有效人店关系。计划表当前日期不是连续集合，进一步说明不能把一个未来计划日期当作常态产能证据。

拜访计划算法会读取按“人员 × 线路 × 日期”和“人员 × 门店 × 日期”形成的组合，以及人员状态，并过滤离职人员。计划生成任务虽然每 30 分钟触发，但成功日期受 Redis 标记跳过；源码中生产最长配置为 12 小时，staging 为 10 分钟。因此“Cron 30 分钟”不等于“任何关系变化 30 分钟内生效”。

4. ttf：人店关系影响模式，模式影响计划与折算

ttf 是门店覆盖折算参数，不是人员数量，也不是拜访次数本身。它依赖门店定义、覆盖模式、店型和人员模式等输入；人店关系可以通过单双人模式影响部分大店 ttf。

因此，人员变更可能影响 ttf，但路径不是：

```
员工状态改变 → ttf 字段自动改变
```

更接近：

```
当前有效人店关系 / BM 服务商
  → 人员模式
  → 店型与覆盖规则
  → ttf / 固定覆盖频次
  → 标签重算与计划输入
```

当前标签中心通过 Kepler 场景和算子重算门店 ttf；这是独立派生链。项目要验证源关系、重算触发、`tag_data.store_tag` 或综合信息投影，以及计划/报表读取的新值，不能只修改人店关系后直接期待 ttf 变化。

5. 审批：组织 assignee 与个人账号不是一回事

门店审批的一级/二级审批人由组织、主管链、渠道与配置解析。当前实现中的任务 `assignee` 可以是组织节点；登录人执行任务时，该组织必须落入其自身或授权组织集合。

从人员角度，审批链要核对：

```
发起人
  → 当前组织 / segment / 门店属性
  → 主管或特定管理组织
  → Flowable 流程版本
  → act_ru_task.assignee
  → 用户可用组织与代理审批角色
  → execute_task
```

2026-10-27 06:43 CST 的 staging 聚合为：

口径	当前值
有效待审门店审批	91 单
与这些流程连接的运行任务	91 任务 / 91 流程
全表运行任务	1,938 任务
全表减当前有效连接	1,847 任务

因此，把 1,938 全部当作业务积压是错误口径。排障应从有效待审 91 单和对应流程实例开始；其余任务需按历史悬挂治理。

同时，“任务存在”“页面能看到”“当前账号能执行”仍是三个断言。桌面端审批查询和企微待办使用不同接口与组织条件；列表可见不代表执行权限成立，跨 segment 和代审批尤其需要正负样本。

6. 报表：人员维度可能来自多个投影

人员进入报表可能表现为：

- 拜访人、负责人、主管或门店归属维度；
- 组织、城市、区域和 segment 汇总；
- 人效、执行率、覆盖频次和渠道考核；
- 人员变更前后的历史归属。

报表需要固定：

```
周期
+ 事实时点
+ 人员 / 组织键
+ 门店集合
+ ttf / 覆盖等计算属性
+ 投影或截P（取明细最后一访的冻结快照）水线
```

如果用今天的组织或负责人回看历史拜访，旧事实可能被重新分配；如果报表读取综合信息投影，而关系 SoT 已更新，两者短期不同不等于谁算错。

当前门店主管投影共 1,693,161 行，但最新更新时间停在 2025-01-24；用户中心 2026P10 主管链为 8,921 行。这直接说明“有主管链”和“所有人员/门店报表使用最新主管链”不是同一事实。

7. 延迟消息只证明 outbox 状态

现有关系修改在生产环境次日 00:00 生效；staging 的修改关系在写入后约 10 分钟生效。变更消息由每日 00:00 的 `store_emp_delay_msg_job` 扫描 outbox 发布，发送后软删，约 8 天后物理清理。

2026-10-27 06:43:35 CST 当前观察：

指标	当前值
延期消息	32 行 / 32 request_id
类型	`ImportStoreEmp` 1 / `UpdateStoreEmp` 31
`is_deleted=1`	32
未处理 `is_deleted=0`	0
涉及 distinct 门店	13
最早 / 最晚创建	2026-10-19 / 2026-10-26

这说明当前保留窗口内的 32 条消息均已软删，不能说明它们对应的综合信息、规则缓存、计划、ttf 和报表全部刷新成功。消息发布、MQ 消费、投影写入和目标业务查询需要各自的日志或回读证据。

8. 一个人员变更的端到端检查顺序

第一步：确认人员身份和业务关系

检查人员类型、在职状态、segment、source、duty、state 和有效期，确认门店关系 SoT 是否到达目标时点。

第二步：确认账号与授权

检查应用账号、客户端、角色、菜单、自身/授权组织和冻结状态。账号有效不代表业务权限正确。

第三步：确认变更消息与投影

检查 `store_emp_delay_msg` 是否发出、软删，以及综合信息、`store_query_*` 或其它目标投影是否在承诺水线更新。

第四步：确认业务关系

根据目的分别检查：

- 线店关系与拜访计划；
- 组织与主管链；
- 审批任务 assignee；
- ttf 重算与门店标签；
- 报表人员/组织维度。

第五步：确认业务结果

用目标真人或脱敏测试身份验证：

- 应该出现的门店/任务出现；
- 不应出现的范围不可见；
- 新计划、审批和报表在指定时点符合预期；
- 历史事实仍可按当时关系解释。

9. 验收矩阵

编号	场景	最低断言
PV-01	新业务员负责门店	SoT、投影、线路/计划、拜访页按顺序可见
PV-02	业务员与负责人不同	两类职责分别消费，不串写
PV-03	修改关系次日生效	生效前旧值、生效后新值均可解释
PV-04	新增关系	关系和消息路径与修改关系分开验证
PV-05	延期消息已发送	消息为软删，下游目标实际刷新
PV-06	主管链调整	新审批任务 assignee 组织正确
PV-07	审批列表与执行	可见、可执行、越权拒绝同时验证
PV-08	单双人模式变化	人店关系、人员模式、ttf、计划和报表逐步回读
PV-09	离职或冻结	新计划/任务不再分配，历史保留
PV-10	同一门店多 source	MW/IC/DA 关系与投影字段不串写
PV-11	主管投影陈旧	不把目标报表误判为实时主管链
PV-12	历史报表重跑	使用期次快照还是当前维度有明确口径

矩阵中的“可见”必须指明页面、接口、资源范围和观察时间；仅截图不能替代数据回读。

10. 四个反例

反例一：页面显示新业务员，所以所有下游已刷新

页面可能读最新 SoT，而拜访计划、ttf 和报表仍读不同投影。

反例二：延期消息已软删，所以业务一定成功

软删只证明 outbox 被处理。没有 MQ 消费和下游查询，不能证明全部消费者成功。

反例三：主管链有数据，所以审批人一定正确

审批还受门店属性、流程版本、组织解析、配置和代理授权影响。任务 assignee 是组织时，登录人可用组织也必须核对。

反例四：报表残留旧人，所以源关系没改

报表可能读取旧截P、旧投影或旧期次。先对齐事实时间与报表水线，再判断源关系。

11. 本章结论与未知

本章可以支持：

1. 人员通过账号、组织、关系、投影和业务结果多层进入执行链；
2. 拜访计划依赖人店、线店、线路和人员状态，不能由一个关系表替代；
3. ttf 是独立派生链，人店关系只是部分输入；
4. 审批任务 assignee 可能是组织，列表可见与执行权限不同；
5. 当前 91 个有效待审流程与 91 个任务相连，全表 1,938 个任务其余 1,847 个不能直接计作积压；
6. 当前保留窗口内 32 条延期消息均已软删，但不证明各下游业务刷新成功。

本章仍不回答：

- 一条已脱敏人员变更案例从 SoT 到拜访、审批、ttf、报表的逐跳时延；
- 当前 32 条延期消息分别被哪些消费者处理；
- 门店主管投影长期停留在 2025-01-24 的业务影响；
- 当前计划表规模为何与有效线店关系差异巨大；
- 报表人员维度的所有投影继承链；
- 冻结、离职和授权回收在全部应用中的实际覆盖；
- staging 与生产的生效时延、权限和数据分布是否一致。

下一章把分散的权限与关系收束到入转调离：一次人员变化如何分别改变账号、权限、业务关系和历史留痕。生命周期字段见案头 A，写链和投影见案头 B，访问矩阵与回收验收见案头 C，尚未关闭的风险见案头 D。

第 11 章 入转调离、账号、权限与关系生命周期

一个人入职、转岗或离开时，页面上的“人员状态”并不会自动完成所有后续动作。企业人员、应用账号、第三方登录、角色、菜单、组织授权、人店关系、主管链和项目责任各有自己的状态与时点。

如果项目只停用了人员或账号，可能仍留下有效门店关系、授权组织、客户端角色或历史任务；如果只关闭人店关系，账号仍可能进入其它应用。入转调离因此不是一条后台事件，而是一组需要并行完成的回收与保留任务。

本章的中心问题是：**一个人的生命周期变化怎样映射到身份、访问、业务关系、执行任务和历史保留？**

1. 七类对象要分别看

对象	典型状态	不能替代
企业人员	`on_job`、离职日期、来源	账号有效
应用账号	`valid`、`frozen`、删除	在职状态
第三方身份	OKTA/企微映射、状态	角色和业务权限
角色/菜单/客户端	启用、关联、分配	数据范围
自身/授权组织	有效期、组织节点、资源	人店责任
人店/主管/客户/经销商关系	状态、职责、有效期	项目成员身份
项目成员	`active`、项目角色、阶段	业务系统账号

当前已有反例：启用客户端可以没有角色，启用角色可以没有客户/菜单/用户，有效账号可以没有角色，人员关系可以审批中，项目成员也可能从未登录业务系统。

2. 入职：从责任登记到真实可用

2.1 项目责任

项目仓 `members.yaml` 只登记此人在项目中承担什么责任、何时参与。它不是 IAM 授权。

2.2 企业身份

按 `employee_code` 检查人员在 `personnel` 与 `mdp_personnel` 中的唯一性、来源和 `on_job`。人员存在不等于目标应用账号存在。

2.3 应用账号

账号键至少是 `app_code + employee_code`，并检查 `valid`、`frozen` 和逻辑删除。登录还需外部 IdP 证明凭证，再按 `redirect` 命中目标应用/客户端。

2.4 第三方身份

OKTA 或企微的绑定是认证映射。绑定存在不等于菜单可见，更不等于数据范围正确。

2.5 功能权限

检查：

```
user_role
  → role
  → role_client
  → role_menu
  → menu
```

只有角色存在不够，必须证明它连接到目标客户端和菜单。

2.6 数据权限

检查 `user_org`、`grant_user_org`、组织资源绑定和门店 segment/覆盖范围。授权组织扩大必须有理由、审批、有效期和回收日期。

2.7 业务关系

若此人实际承担门店责任，还需建立 `store_emp` 关系并验收生效时间；人店关系、线店关系和主管链分别检查。

3. 新项目访问矩阵

每位需要进入业务系统的成员应有一行：

字段	内容
人	姓名、工号、项目角色
应用/客户端	目标入口
认证	OKTA、企微等
业务角色	必要角色
菜单	最小功能集合
自身组织	归属
授权组织	额外范围、理由与期限
生效/回收	起止日期
正样本	应可访问的对象
负样本	应不可访问的对象
Owner	业务与访问责任人

矩阵不是权限本身；它把责任、配置和验收样本连接起来。

4. 转岗：不是替换一个组织字段

转岗可能同时改变：

- 人员 segment 与主管；
- 自身组织；
- 授权组织；
- 应用/客户端角色；
- 门店职责 L/F；
- 线路与计划；
- ttf 人员模式；
- 审批人解析；
- 报表人员维度。

项目应比较变更前后：

```
person
+ accounts
+ roles/menus
+ organizations/grants
+ store relations
+ planning/approval consumers
+ reports
```

只更新 `user_org` 可能让角色仍指向旧职责，也可能使有效人店关系和新组织范围矛盾。

5. 离职：四种回收并行

5.1 企业人员

上游 `on_job` 变化是人员状态事实，但不能假设所有下游已消费。

5.2 账号与认证

检查账号 `valid` / `frozen`、第三方身份绑定和应用客户端访问。账号冻结与人员离职是不同记录。

5.3 权限与组织

回收角色、临时菜单、客户端关联、授权组织和资源范围。审计历史保留。

5.4 业务关系与执行

检查人店关系关闭、主管链刷新、拜访计划排除、审批任务重分配或取消、报表历史保留。

当前人店关系验收记录显示：静态接入检查通过，但真人写入 E2E 尚未执行，因此不能把 staging 配置存在写成离职回收已通过。

6. 生效时间必须分层

当前人店关系已有明确差异：

- 生产修改关系按次日 00:00 生效；
- staging qa 修改关系约当前 +10 分钟；
- 延期消息仍由每日 00:00 任务扫描；
- 综合信息、计划和报表读取不同投影。

因此：

```
人员状态时间
≠ account 状态时间
≠ relation SoT 生效时间
≠ delay message 发送时间
≠ projection / report 刷新时间
```

项目必须分别记录实际时间，不能用一个“T+1”覆盖全部链。

7. 历史保留与最小必要

离职回收不等于删除全部历史：

- 历史审批、拜访、订单和报表要保留业务解释；
- 账号、设备和审计记录按安全政策保留；
- 姓名、电话、身份证明等个人信息不进入无关报告；
- 测试和案例优先使用聚合或脱敏业务键；
- 人店关系关闭后历史区间仍可回答“当时谁负责”。

清理策略必须区分业务历史、审计留存、临时测试数据和个人信息。

8. 生命周期验收矩阵

编号	场景	必须断言
PL–01	新项目成员仅做文档	不自动创建业务账号
PL–02	新业务用户入职	人员、账号、IdP、角色、菜单、数据范围逐层通过
PL–03	授权组织临时扩大	理由、期限、负样本和回收完整
PL–04	入职即承担门店责任	人店关系 state/effective time 正确
PL–05	转岗改变 segment	组织、角色、门店关系和下游一致
PL–06	转岗只改组织	检查残留角色和旧业务关系
PL–07	离职账号冻结	客户端不可进入，历史保留
PL–08	离职关系关闭	新计划/审批不再分配，旧事实保留
PL–09	离职授权回收	范围外数据不可见
PL–10	批量导入	重复提交幂等，审批中关系不被误用
PL–11	审批人变更	新任务使用正确组织，旧任务按规则处理
PL–12	回收后审计	操作人与变更历史可追溯

9. 反例

反例一：项目成员名单就是账号清单

名单只证明项目责任。业务账号仍需身份、应用和权限链。

反例二：账号冻结等于人员关系关闭

账号、人员状态和门店关系是不同对象。历史关系可能仍有效。

反例三：人员离职后所有历史删除

历史事实需要保留与审计，删除会破坏报表和责任解释。

反例四：只验菜单可见

数据范围和负样本未验，不能证明授权正确。

10. 本章结论与未知

本章可以支持：

1. 入转调离要分别处理人员、账号、认证、权限、组织、业务关系和项目责任；
2. 项目成员不是业务账号；
3. 人店关系自身有时间窗、状态和延迟消费；
4. 离职回收与历史保留需要同时设计；
5. 员工关系当前 staging 写入型 E2E 仍未完成。

本章仍不回答：

- 所有人员变更事件的生产端到端时延；
- 所有角色和应用的离职回收是否自动；
- 项目成员与业务账号的自动对账方案；
- 每条个人信息字段的完整保留期限；
- 主管链陈旧时的生产实际影响；
- 当前 staging 的回收结果能否外推生产。

下一章把这些生命周期动作收束成人员变更影响图、项目开通流程和运维证据顺序，回答问题发生时怎样定位到正确的层。变更卡与字段见案头 A，消费链见案头 B，项目开通和验收见案头 C，现行缺口见案头 D。

第 12 章 人员变更影响、项目开通与运维演进

人员问题的工单常从一句模糊现象开始：“他看不到”“审批没过来”“报表还是旧人”。如果值班人员先重置账号或直接改关系，可能修错层，甚至留下新的越权和历史断点。

可靠定位必须沿“人员 → 账号 → 权限 → 关系 → 投影 → 行为/报表”链逐层排除。项目开通也遵循同一逻辑：项目责任、企业身份、登录、功能、数据范围和业务关系分别成立。

本章的中心问题是：**人员变更和访问问题怎样形成可执行的影响图、验收包与运维闭环?**

1. 人员影响链

```
Personnel
  → User / Account / IdP
  → Organization / Supervisor
  → Role / Client / Menu
  → Own / Granted Organization
  → Store / Route / Customer / Distributor relation
  → Plan / Approval / ttf / Report
```

每个箭头都要问三个问题：

问题	示例
靠什么键连接	employee_code、user_uid、org uid、store_code
何时生效	人员状态、账号状态或关系有效期
谁读取	页面、规则、计划、任务、报表

只画箭头不写键和时点，影响图无法验收。

2. 问题分诊

现象	先查	再查
无法登录	IdP 凭证、账号 valid/frozen、人员 on_job	redirect、client、应用入口
能登录看不到菜单	user_role、role_client、role_menu	菜单启用与前端路由
看得到菜单看不到数据	自身/授权组织、资源、segment	门店/报表目标入口规则
看得到范围外数据	入口是否绕过数据权限	角色、组织、查询 SQL/API
门店看不到新业务员	关系 SoT 生效时间	延期消息、投影、计划/报表
审批没有待办	是否命中审批、页面接口	当前审批行、任务 assignee、组织
页面能看到但不能审	当前任务 assignee	自身/授权组织、代审批角色
报表仍是旧人	报表读取期次/截P/投影	人员/组织关系当前值
离职后仍可访问	账号和 IdP	角色、组织、客户端与缓存

先判断问题属于身份、功能权限、数据权限、业务关系还是报表时点。

3. 一次人员变更的影响卡

变更单至少记录：

项	内容
人员	employee_code（输出按最小必要脱敏）
变更类型	入职、转岗、离职、组织、主管、门店责任
新旧值	人员、账号、角色、组织、关系各层
生效时间	各对象自己的时间
直接消费者	账号、计划、审批、ttf、报表
正样本	应可见/应收到
负样本	不应可见/不应收到
历史策略	保留、关闭、重算或快照
回退	角色、组织、关系和缓存如何恢复
Owner	业务、访问、数据和应用责任

变更卡只记录配置前后值还不够，要写清楚哪一层已完成、哪一层仍待调度。

4. 项目开通：从责任到真实登录

4.1 先选接入方式

- 复用已有应用/客户端；
- 增加角色/菜单/数据范围；
- 新入口需要新客户端；
- 独立认证边界才新建应用。

项目不是用户中心实体。默认不为每个项目新建应用。

4.2 建立最小访问矩阵

矩阵必须包含目标应用、客户端、认证方式、角色、菜单、自身组织、授权组织、有效期和正负样本。

4.3 做前置检查

人员唯一、在职、账号唯一、有效、未冻结、第三方绑定和 redirect 唯一命中。

4.4 做真人验收

目标本人从真实 redirect 登录；范围内可见/可操作，范围外不可见/不可操作。敏感能力另验角色、组织和代理授权组合。

5. 审批与拜访中的特殊风险

5.1 审批 assignee 是组织

当前任务 assignee 可能是组织节点，不是个人账号。执行权限要求组织在登录人的自身或授权组织内。列表接口可见不等于任务可执行。

当前 staging 有效待审 91 单连接 91 个任务，全表另有大量历史悬挂任务。运维必须以有效审批行连接任务，不能用 `act_ru_task` 全表数量当积压。

5.2 拜访计划不是人店关系的实时副本

计划还依赖线路、人员状态、覆盖模式和调度 TTL。关系可见后，计划仍可能未生成；计划可见也不代表员工已执行。

5.3 ttf 和报表使用派生值

人店关系只是人员模式的部分输入。ttf 重算、门店标签、计划密度和报表维度都要单独回读。

6. 运维证据顺序

6.1 环境与版本

记录 Deployment、镜像、profile、页面版本和数据库观察时间。

6.2 人员与账号

只查看目标人员所需字段，避免输出无关个人信息。

6.3 权限

检查角色到客户端/菜单、自身/授权组织与资源。

6.4 业务关系

检查 `source`、`duty_type_code`、`state` 和有效窗口。

6.5 消息与投影

检查延期消息、MQ、综合信息、`store_query_*` 和目标应用。

6.6 行为与报表

用正负样本证明计划、审批、ttf 和报表实际结果。

7. 当前运维缺口

缺口	影响
延期消息软删不证明下游成功	需要 MQ 与目标回读
门店主管投影停在 2025-01-24	主管/审批/报表可能使用旧链
多应用、多身份映射未统一对账	离职可能残留访问
项目成员与账号无自动映射	新项目开通需人工矩阵
部分启用角色/客户端无完整关系	配置存在不等于可用
授权组织来源与回收不完整	越权与长期残留风险
人店关系 staging 写入型 E2E 未执行	不能签生产上线
ERTM、合作方关系分工未闭合	外部人员模型仍需裁定

这些缺口的处理应由业务、IAM、应用、数据和安全 Owner 共同完成。

8. 监控与指标

面	建议观察
身份	人员来源、在职、账号有效/冻结、绑定唯一
功能权限	角色无客户端/菜单、账号无角色
数据权限	无组织、资源无法解析、范围外命中
人店关系	空工号、重复职责、审批中、未来行
消息/投影	未发送 outbox、失败消费、投影滞后
计划/审批	未生成计划、任务总表与有效连接差异
报表	旧人员维度、组织快照与刷新水线
回收	离职后账号、角色、授权、关系残留

阈值需基于生产基线和业务风险，不在 staging 计数上发明。

9. 隐私与安全

人员手册的案例和排障界面必须遵守：

- 只保留验证所需工号和业务对象；

- 不展示无关姓名、手机号、身份证明和地址；
- 不记录密码、Token、Cookie、API Secret 或 Jasypt 值；
- 不使用真人身份代替测试；
- Agent不得伪造 `X-USERNAME`、复用真人凭证或自行授角色；
- 访问日志、审计日志与业务历史分别遵循保留策略。

“为排障方便”不是扩大个人信息读取范围的理由。

10. 反例

反例一：先重置账号

如果根因是数据范围或关系投影，重置账号不会解决，还会产生额外变更。

反例二：全表任务当审批积压

历史悬挂任务会夸大 SLA。先连接当前有效审批。

反例三：延期消息已删除就是全链成功

软删只证明 outbox 被处理。需要 MQ 和目标消费回读。

反例四：Agent 可独立开通

当前没有 Agent 用户类型、角色和项目授权模型。必须由人类委托人承担和批准。

11. 本章结论与未知

本章可以支持：

1. 人员问题应沿身份、权限、关系、投影和结果链定位；
2. 项目开通不是新建一个项目或应用，而是建立责任与访问矩阵；
3. 审批、人店、计划、ttf 和报表各有消费边界；
4. 延期消息和历史任务不能用单一数量解释；
5. 当前存在主管链、IAM 回收和员工关系 E2E 等明确缺口。

本章仍不回答：

- 当前所有应用的离职自动回收覆盖率；
- 项目成员与账号的最终自动对账方案；
- 主管链、计划和报表的真实生产端到端时延；
- Agent 是否应成为正式身份；
- 个人信息全部字段的留存期限；
- staging 结论与生产状态的差异。

最终回到全书主线：先判断问题落在人员、账号、权限、业务关系、投影还是业务结果，再沿证据链逐层验收。查对象和字段回到案头 A，查系统与数据链回到案头 B，查项目开通回到案头 C，查缺口和隐私边界回到案头 D。

第 A 卷 人员对象、字段与关系

A.1 对象总表

对象	回答	典型键	不能替代
Personnel	企业人员是谁	employee_code	应用账号
UserInfo	某应用下的账号	app_code + employee_code	人员在职
Third-party identity	外部 IdP 身份	app_code + employee_code + thirdPart	角色
OrganizationContent	组织树节点	org_content_uid	数据资源
UserOrg	用户自身组织	user_uid + org node	授权组织
GrantUserOrg	用户额外组织	user_uid + org node	业务责任
Role	业务角色	app_code + role_code	用户
Client	登录入口	client code	应用
Menu	功能菜单	menu uid	数据范围
StoreEmpRelation	门店责任	store + emp + duty + source + time	主管链
SupervisorRelation	向谁汇报	employee + period/source	门店责任
Cooperate/DistributorPerson	外部业务人员映射	source + person/customer/dealer key	Mars 员工账号

A.1.1 常用缩写

写法	本书中的含义	不应直接推成
MDP	Mars 人员、组织与相关主数据的上游来源	某一个应用账号已经开通
DI	数据接入、落地区与加工层的统称；具体表名以证据为准	最终业务人员表
SFA	Sales Force Automation，销售执行应用族；当前常见应用编码为 `cotsfa` 与 `dasfa`	人员类型或业务角色
AD	企业目录账号；当前 OKTA 入口使用 AD 号定位用户	业务系统角色
DSR	`mdp_personnel.employee_type_name` 中的一种人员类型	某个应用角色或门店职责
DA	人员类型名称，同时存在 `dasfa` 应用入口；两者不是同一个维度	自动拥有 DA SFA 菜单或门店范围
BM	BM 外包服务商及其人员在门店覆盖中的业务关系	Mars 正式员工账号
COT	一线作业入口与业务应用族，也是人店关系的来源之一	所有应用账号的总称
IC	冰淇淋业务 segment；在人店关系中也可作为来源值出现	某个人员角色
MW	糖巧业务 segment；在人店关系和投影字段中常见	全公司统一门店范围
IAM	身份与访问管理，包括账号、第三方身份、角色、组织和回收	项目成员责任
IdP	外部身份提供方，如 OKTA 或企业微信	业务功能权限
SoT	Source of Truth，事实源或权威写入源	报表、综合信息或查询投影
ttf	门店覆盖折算参数，受店型、分层、单双人模式和覆盖规则影响	人员数量或拜访次数
截P	取明细最后一访形成的冻结快照口径	当前实时明细
E2E	End-to-End，从入口、配置、事实源到下游结果的端到端验证	单个接口返回成功
MQ	消息队列；本书用它指变更消息的异步传递与消费	目标业务已经刷新
SLA	对处理或刷新时延的服务承诺；必须区分关系生效、消息发送和下游刷新	所有环节使用同一时延
TTL	缓存或派生数据的生存时间，过期后会按既定策略重算或失效	永久有效的关系
A1	门店建店或修改的审批阶段；本书只使用它串联人店关系状态	人店关系自己的业务主键
ERTM	当前证据中尚未闭合的外部关系或收货方类型代码	已确认的合作方人员对象

这些缩写只用于降低检索门槛。具体对象、字段和当前实现仍以对应正文、案头卷和 `source_docs` 为准。

A.2 人员

`personnel` 是用户中心消费/加工表；`mdp_personnel` 是 MDP 镜像。两者都有 `employee_code`，但当前精确交集不能按总行数相减解释。

人员卡字段：

字段组	字段
身份	employee_code、employee_name、source_code/name
用工	employee_type_code/name
在职	on_job、on_job_date、quit_job_date
主管	supervisor_code/name、linemanager
地理	mars_region/province/city_cluster/city
外部关联	dt_id、customer_name
渠道技术字段	ad_number、email

人员卡默认不展示无关手机号、证件和地址。

A.3 用户与认证

`user_info` 关键字段：

字段	含义
`uid`	用户唯一标识
`app_code`	应用
`employee_code`	工号
`account`	登录账户
`idp_source`	IdP 来源
`frozen`	是否冻结
`valid`	是否有效
`effective_*`	拉链有效期

第三方身份在 `user_thirdparty_info`，至少看 `app_code + employee_code + third_part`。

认证成功要求：

```
外部 IdP 完成凭证验证
+ redirect 唯一命中 client
+ user_info 有效且未冻结
+ personnel u mdp_personnel 在职且唯一
```

A.4 组织与资源

对象	关键字段
`organization`	uid、name、app_code、状态、有效期
`organization_content`	uid、parent_uid、org_uid、level、uid_path
`user_org`	user_uid、org_content_uid
`grant_user_org`	user_uid、org_content_uid、授权来源
组织资源绑定	org node → resource category/node

组织节点可见不等于资源已解析；授权组织不等于门店数据范围正确。

A.5 角色、客户端与菜单

功能权限链：

```
user_role
  → role
  → role_client
  → role_menu
  → menu
```

检查：

- 角色是否属于目标应用；
- 角色是否连接目标客户端；
- 角色是否包含目标菜单；
- 菜单是否启用；
- 用户是否有效；
- 入口是否还要求代理/审批标志。

A.6 人店关系

最小事实：

```
store_code
+ emp_code
+ duty_type_code
+ source
+ state
+ effective_start_timestamp
+ effective_end_timestamp
+ is_deleted
```

职责：

duty	含义
L	业务员
F	门店负责人

来源当前观察包括 COT、IC、DA。不同 source 的关系不能仅按 store+emp 去重。

当前有效：

```
state=1
AND effective_start <= target_time
AND effective_end > target_time
AND is_deleted=0
```

A.7 主管与其它关系

关系	载体
用户期次主管	`di_otr_user_org_supervisor_*`
门店主管投影	`store_emp_supervisor`
合作方人员	`org_cooperate_user` / `cooperate_user_mapping`
经销商人员	`distributor_emp*` 多库副本
项目责任	项目仓 `members.yaml`

这些关系的键和用途不同。同名表存在多个库副本时，也不能直接合并。

A.8 状态与时间

状态/时间	载体
人员在职	`on_job`
账号有效/冻结	`valid` / `frozen`
人店审批状态	`state=1/2/4`
关系生效	effective window
延期消息	create/type/is_deleted
组织/主管期次	build_time/period

“人员停用”“账号冻结”“关系关闭”“投影刷新”是四种完成状态。

A.9 最小关系卡

项	内容
人员	employee_code
应用	app_code
账号	user_uid
组织	own/grant node
功能	client/role/menu
业务关系	store/customer/dealer + duty/source
时点	effective/observation
证据	配置、登录、查询、行为

A.10 禁止混用

- Person、User、Account、Role；
- 项目成员、业务账号；
- menu 可见、数据可见；
- own org、grant org、store scope；
- store manager、salesman、supervisor；
- 合作方、经销商人员、项目成员；
- 当前投影、历史事实。

第 B 卷 人员系统、身份、权限与数据链

B.1 身份主链

```
MDP
  → personnel / mdp_personnel
  → user_info
  → user_thirdparty_info
  → auth
  → app_content / client
  → user_role / role_client / role_menu
  → user_org / grant_user_org
  → resource binding
```

人员、账号、认证、功能和数据范围分层建立。

B.2 用户中心运行面

Deployment	主要职责
yhx-master	人员、组织、账号、角色、授权和查询
yhx-idp	登录与第三方身份
cuckoo-idp	身份接入
cuckoo-locust	查询运行面
yhx-page	用户中心页面

2026-10-26 均为 1/1。就绪不等于登录、角色或数据范围通过。

B.3 认证链

```
User
  → external IdP (OKTA / 企微)
  → redirect
  → unique client
  → user_info states
  → personnel states
  → business access
```

`/auth/check/*` 不验证密码；密码门应由外部 IdP 完成。

`/biz/**` 与 `/open/**` 的拦截边界不同。`/open/**` 不得直接暴露浏览器或外网。

B.4 功能权限与数据权限

层	入口
功能	user_role → role → client/menu
自身组织	user_org
授权组织	grant_user_org
资源	organization_content_category_node
门店范围	segment + geography + coverage + entry rule

功能验收与数据验收必须分开。

B.5 人店关系链

```
COT/IC/DA/导入
  → re-sops /store_emp/*
  → store_ops.store_emp
  → store_emp_delay_msg / MQ
  → zhx 综合信息
  → store_query_* / 拜访 / ttf / 审批 / 报表
```

四条写入路径：

1. COT 建店，随 A1 状态联动；
2. COT/IC 修改，环境决定生效时点；
3. DA 直接新增；
4. 公海/IC 批量导入，同步调用服务。

B.6 主管与审批

审批链：

```
user + org + segment + store context
  → lineManager / KAM / cityManager / channelManager
  → Flowable process
  → act_ru_task.assignee
  → user own/grant org
  → execute_task
```

任务 assignee 可以是组织。页面可见与可执行分开验。

B.7 拜访、ttf 与报表

消费面	链
拜访	人店 + 线店 → plan → visit
ttf	人店/BM → 人员模式 → 店型/覆盖 → tag → plan
审批	组织/主管 → assignee → flow
报表	人员/组织/门店键 + period + projection

关系存在不代表计划、ttf 或报表已刷新。

B.8 外部和经销商人员

关系	当前载体
合作方组织人员	`org_cooperate_user`
合作方映射	`cooperate_user_mapping`
经销商人员	`store_mg_xg.*` / `yyx_da_master.*` / `yyx_icsfa_master.*`

同名表存在多库副本；必须声明数据库、系统和业务用途。

B.9 证据模板

```
环境 / 时间
人员 employee_code
app_code / client
user_uid / third_part
```

```
org node / resource
role / menu
store / duty / source
state / effective
projection / downstream
observed result
```

B.10 只读排查顺序

1. 人员唯一与在职。
2. 账号有效、冻结、IdP 绑定。
3. redirect/client。
4. role–client–menu。
5. own/grant org 与资源。
6. 门店/线路/主管关系。
7. 延期消息和投影。
8. 目标行为或报表。

B.11 当前证据缺口

- MDP 六类同步的逐实体新鲜度。
- `personnel` 与 `mdp_personnel` 精确工号连接结果。
- 主管投影刷新责任。
- 外部合作方模型分工。
- 多应用离职回收自动覆盖。
- 人店关系写入型 E2E 的真人证据。

第 C 卷 人员项目与访问工作台

C.1 访问矩阵

每人一行：

字段	内容
人	姓名、工号、项目角色
系统入口	application + client
认证	OKTA / 企微
功能	role + menu
数据	own org + grant org + segment
业务关系	store/customer/dealer
有效期	start/end
正样本	应可见/可操作
负样本	应不可见/不可操作
Owner	业务、IAM、数据

C.2 新项目开通

```
members.yaml
  → 是否真的需要业务账号
  → 人员唯一与在职
  → app + client
  → IdP 绑定
  → role + menu
  → own/grant org + resources
  → 业务关系
  → 真人正负样本
```

项目成员不等于系统用户。只有需要进入业务系统的人才进入 IAM 链。

C.3 应用/客户端决策

场景	决策
复用现有系统	复用应用/客户端，增角色/范围
新 URL 同应用	评估新 client
独立认证边界	新应用 + client + 角色
一次查询/导出	复用受控入口，不创建 Agent 身份

C.4 责任人变更

提交：

- 门店编码；
- 旧/新人员；
- duty L/F；
- source；
- state；
- effective time；
- request_id/task_uid；
- 前后拉链；
- 延期消息；
- 计划/审批/ttf/报表回读。

C.5 批量导入

检查：

1. 文件模板与 segment；
2. 人员有效/冻结/在职；
3. 门店数据范围；
4. 重复门店/重复提交；
5. 200 行切片调用；
6. store_emp 拉链；
7. 下游刷新。

批量任务成功只证明文件处理链结束，不证明业务关系有效。

C.6 离职回收

面	动作
人员	on_job/离职日期
账号	valid/frozen
IdP	绑定停用/清理
角色/客户端	回收
授权组织	回收
门店/客户/经销商关系	关闭
计划/审批	不再分配/任务处理
历史	保留审计与业务事实

C.7 越权负样本

至少：

- 菜单无权限；
- 角色有但无 client/menu；
- 组织外门店；
- segment 不匹配；
- 授权组织过期；
- 冻结/离职人员；
- 历史审批任务；
- 直接调用内部 `/open/**` 或服务接口。

C.8 人店关系 E2E 清单

当前 staging 记录要求真人执行 E2E-01～14：

1. 糖巧/冰淇淋登录与菜单；
2. 范围内查询/导出；
3. 批量导入；
4. 修改关系；
5. 同人 L/F；
6. 无组织范围；
7. 错误 segment；
8. 冻结/离职；
9. 范围外门店；
10. 重复提交；
11. 双覆盖 MW/IC；
12. 审批中/驳回关系；
13. 未授权调用；
14. 下游刷新。

C.9 证据包

```
访问矩阵
人员/账号/IdP
role/client/menu
own/grant org
resource nodes
```

```
store relations
request_id/task_uid
前后拉链
delay message/MQ
projection
visit/plan/approval/ttf/report
回收与审计
```

C.10 上线条件

- 项目访问 Owner 明确；
- 人员、账号、IdP、角色、数据范围逐层通过；
- 人店关系矩阵与业务 Owner 签核；
- 正负样本由目标真人执行；
- 延迟消息和下游有回读；
- 离职/回收路径可演练。

第 D 卷 人员事实缺口、隐私与维护

D.1 当前缺口

编号	缺口	影响
U-OQ-01	MDP 到用户中心逐实体同步时效	人员新鲜度
U-OQ-02	`personnel` 与 `mdp_personnel` 工号交集/来源	重复与离职
U-OQ-03	门店主管投影停在 2025-01-24	审批/督导/报表
U-OQ-04	授权组织来源与回收	越权和残留
U-OQ-05	多应用离职回收覆盖	安全
U-OQ-06	项目成员与账号自动对账	新项目开通
U-OQ-07	合作方两类关系分工	外部人员
U-OQ-08	经销商人员多库副本	映射解释
U-OQ-09	人店关系写入型 E2E 未执行	生产上线
U-OQ-10	AI Agent 正式身份不存在	委托与责任

D.2 术语

规范概念	不等于
Person / Personnel	User、Role
User account	人员在职
Third-party identity	业务权限
Application	Project
Client	Application
Role	Data scope
Menu visible	Data visible
Own organization	Granted organization
Store manager / salesperson	Supervisor
Project member	Business account
Distributor employee	Mars employee
Current projection	Historical fact

D.3 当前基线

2026-10-26/27 staging：

项	值
user_info	9,009
personnel	412 current nondeleted
mdp_personnel	53,757 current nondeleted
enabled applications	5
current clients	17
roles	44 / 41 enabled
enabled menus	210
user_role	9,824
role_client	49
role_menu	604
organization_content	8,794
user_org	8,751
grant_user_org	482
org resource bindings	115,455
P10 supervisor users	8,921
store supervisor projection	1,693,161；latest 2025-01-24
delay messages	32；全部软删
valid store approvals	91
act_ru_task all	1,938

数值只代表对应 staging 时点。

D.4 隐私规则

允许：

- 聚合计数；
- 目标验证所需工号；
- 脱敏门店/角色样本；
- 业务 Owner 批准的访问矩阵。

禁止：

- 输出无关姓名、手机号、证件、地址；
- 输出密码、Token、Cookie、API Secret、Jasypt；
- 用真人凭证做自动测试；
- Agent 伪造 `X-USERNAME`；
- 以截图替代权限与数据回读。

D.5 事实分级

证据	可证明
人员表	人员记录
user_info	某应用账号
user_thirdparty_info	IdP 绑定
role/client/menu	配置链
user_org/grant	组织绑定
store_emp	门店关系
页面/API	当前配置返回
真人验收	实际可访问与范围

配置存在不等于真人可用。

D.6 维护触发

- MDP 同步或人员模型；
- user/auth/app/client；
- role/menu/resource；
- store_emp 写入与时延；
- supervisor 期次/投影；
- external cooperation / distributor mapping；
- access 开通/回收/OQ；
- staging 或生产基线变化。

D.7 脱敏案例模板

```
案例类型：
脱敏人员键：
业务目的：
应用/客户端：
角色/组织范围：
业务关系：
观察时间：
```

正样本结果：

负样本结果：

证据来源：

未核验：

D.8 维护原则

1. 一个事实只在一个主数据/关系源定义；
2. 投影不反写事实；
3. 角色、组织和业务关系分别审批；
4. 变更带 request_id 和前后值；
5. 个人信息最小必要；
6. 未证实的离职回收不得写成完成；
7. Agent 只作为人类委托人的辅助执行者。

生成边界

本文件是装配成品，不新增业务事实。修改章节、案头卷或 manifest 后必须重新运行生成器；事实冲突回 source_docs 和 claim ledger。